



Python

Table of Content:

- Introduction
- Installation
- First Program
- Variable
- Data Type
- Type Conversion
- Python String
- Print Function
- Python Booleans
- Python operator
- Comment
- Python Input
- Python Collection(List, Tuple, Set, Dictionary)
- If...Else

Table of Content:

- Python Loop (While,For)
- Python Statements
- Range Function
- Function
- Lambda
- Array
- Math Function
- Class
- Inheritance
- Iterators
- Datetime
- Exception
- Project

1. Introduction:

Python is an interpreted [high-level general-purpose programming language](#). Its design philosophy emphasizes code readability with its use of significant indentation. Its language constructs as well as its object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

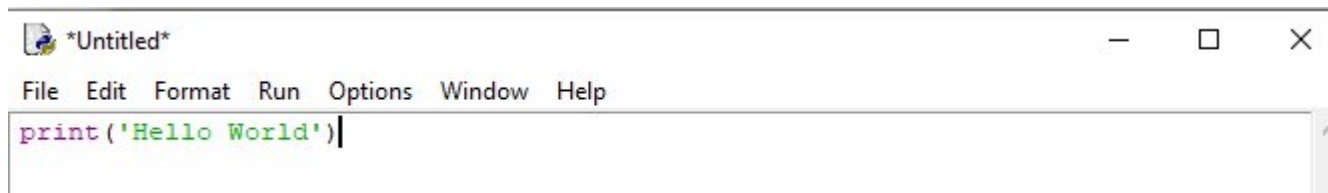
2. Installation:

Live

3.First program:

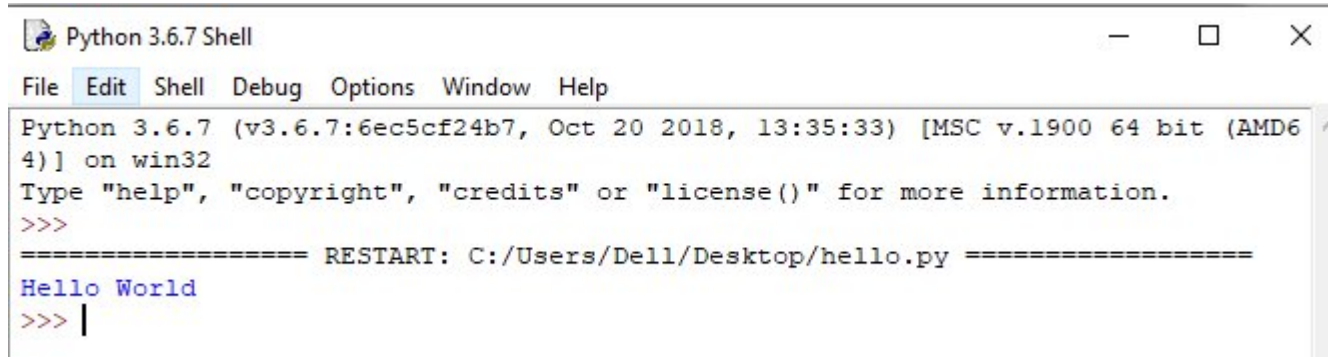
The print() function prints the specified message to the screen, or other standard output device.

Input:



A screenshot of a code editor window titled '*Untitled*'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code area contains a single line of Python code: `print('Hello World')`. The cursor is at the end of the line.

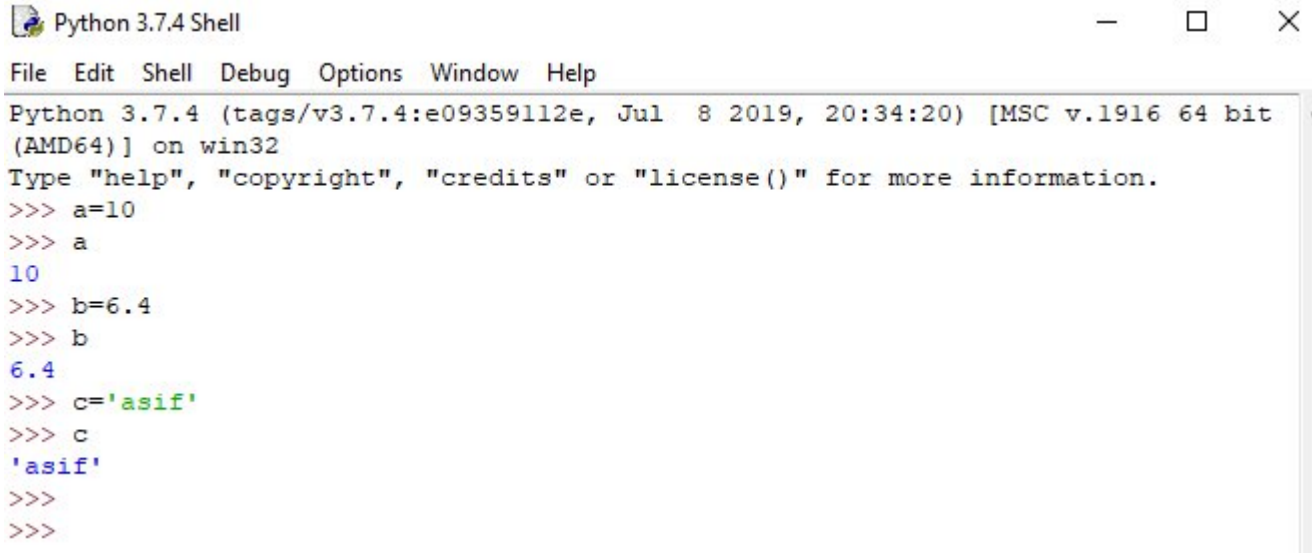
Output:



A screenshot of a Python 3.6.7 Shell window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The text area shows the following output:
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/hello.py =====
Hello World
>>> |

4. Variable:

A Python variable is a reserved memory location to store values. In other words, a variable in a python program gives data to the computer for processing.



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul  8 2019, 20:34:20) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> a=10
>>> a
10
>>> b=6.4
>>> b
6.4
>>> c='asif'
>>> c
'asif'
>>>
>>>
```

❑ Assign Multiple Value:

Input:

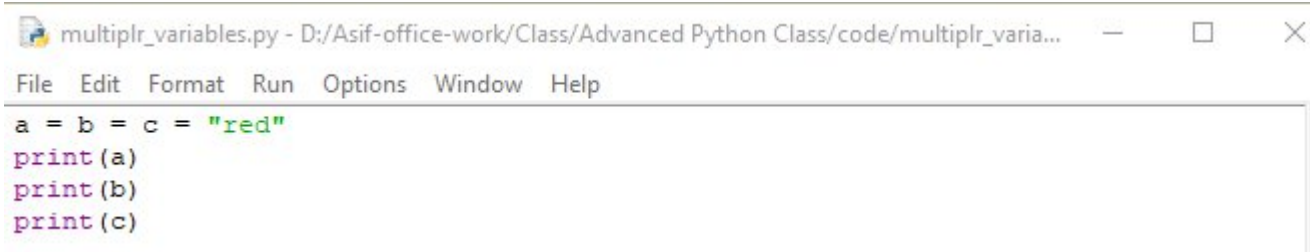
```
multiplr_value.py - D:/Asif-office-work/multiplr_value.py (3.10.4)
File Edit Format Run Options Window Help
a, b, c, d= "red", "Blue", "yellow", "green"
print(a)
print(b)
print(c)
print(d)
```

Output:

```
IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929
64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:/Asif-office-work/multiplr_value.py =====
>>>
red
Blue
yellow
green
>>>
```

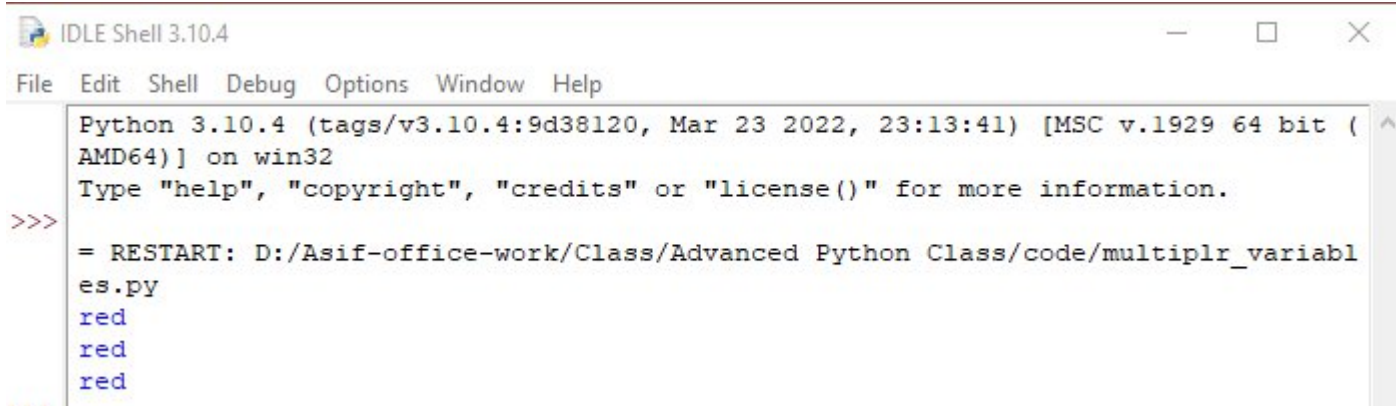
❑ One Value to Multiple Variables:

Input:



```
multiplr_variables.py - D:/Asif-office-work/Class/Advanced Python Class/code/multiplr_varia...  
File Edit Format Run Options Window Help  
a = b = c = "red"  
print(a)  
print(b)  
print(c)
```

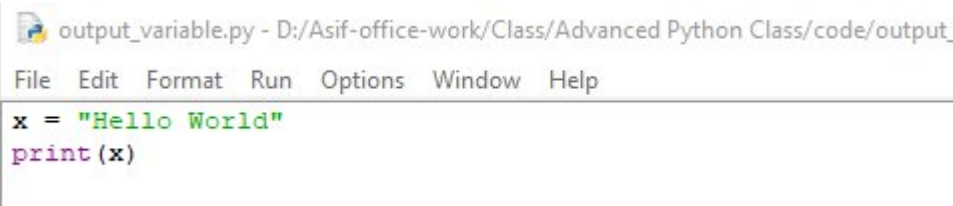
Output:



```
IDLE Shell 3.10.4  
File Edit Shell Debug Options Window Help  
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/multiplr_variab...  
es.py  
red  
red  
red  
...
```

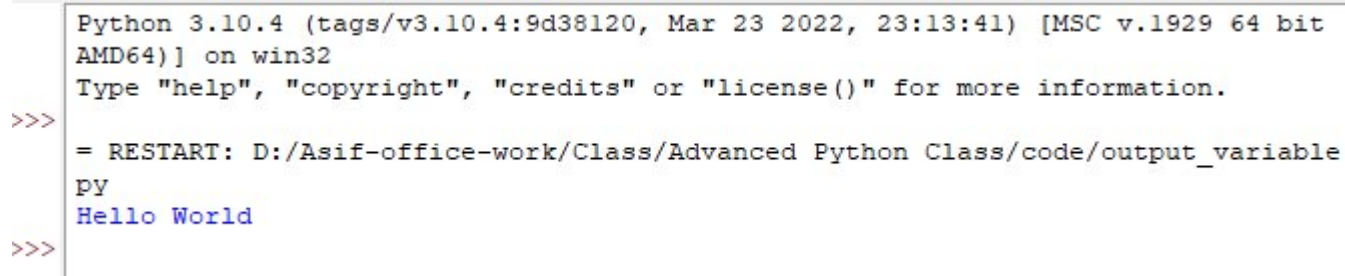
❑ Output Variables:

Input:



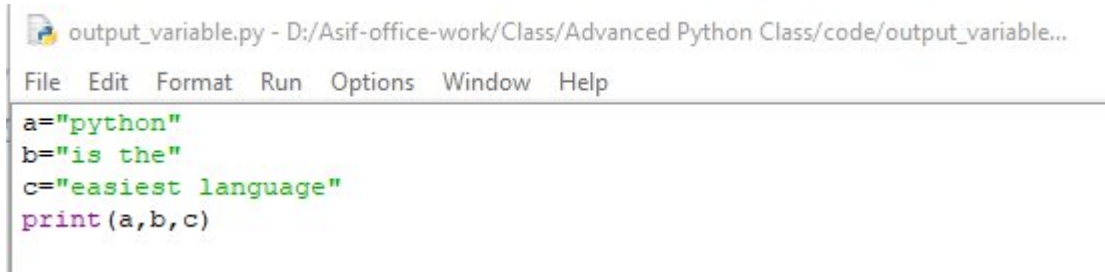
```
output_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/output_
File Edit Format Run Options Window Help
x = "Hello World"
print(x)
```

Output:



```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/output_variable
PY
Hello World
>>>
```

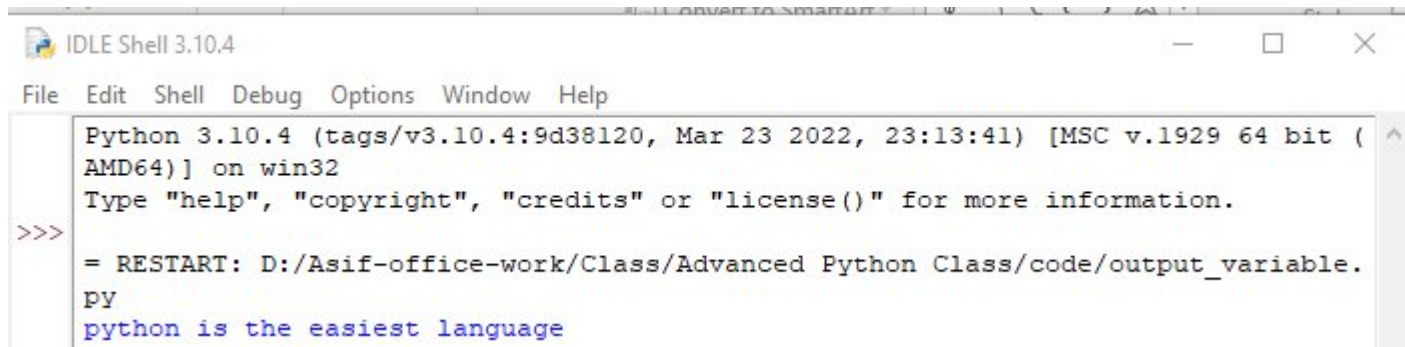
Input:



A screenshot of a Python script file named `output_variable.py` located at `D:/Asif-office-work/Class/Advanced Python Class/code/output_variable...`. The script contains the following code:

```
a="python"  
b="is the"  
c="easiest language"  
print(a,b,c)
```

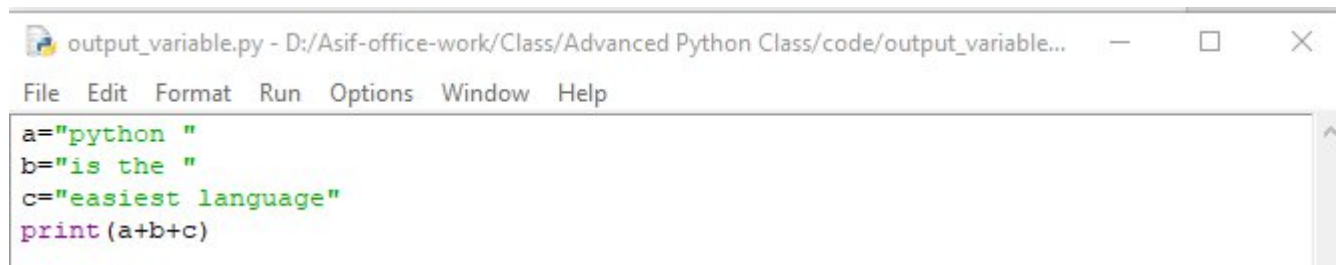
Output:



A screenshot of the IDLE Shell 3.10.4 window. The window title is "IDLE Shell 3.10.4". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The shell displays the following text:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
  
>>>  
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/output_variable.py  
python is the easiest language
```

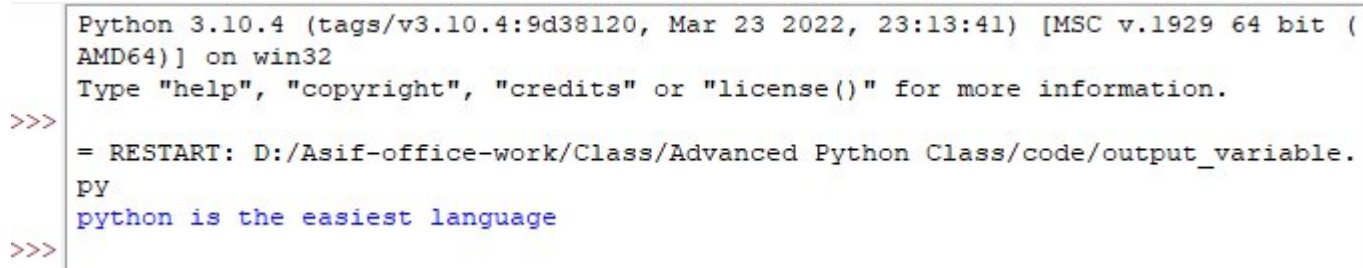
Input:



A screenshot of a Python IDE window. The title bar shows the file path: `output_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/output_variable...`. The menu bar includes `File`, `Edit`, `Format`, `Run`, `Options`, `Window`, and `Help`. The code editor contains the following Python code:

```
a="python "  
b="is the "  
c="easiest language"  
print(a+b+c)
```

Output:



A screenshot of a Python terminal window. The output shows the Python version and environment information, followed by the execution of the script:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/output_variable.PY  
python is the easiest language  
>>>
```

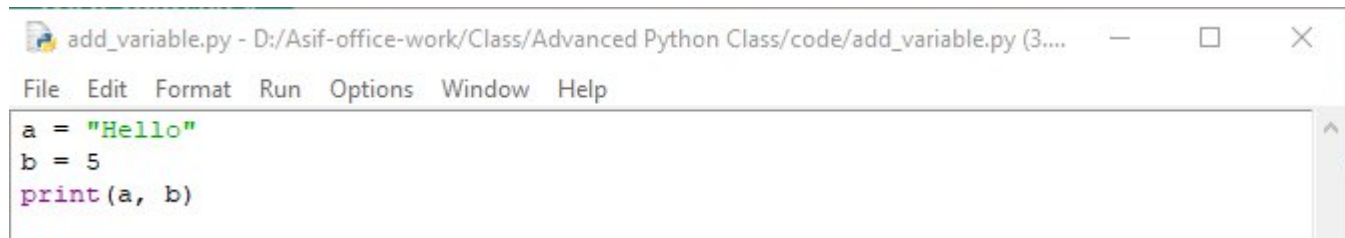
Input:

```
a = 20  
b = 12  
print(a + b)
```

Output:

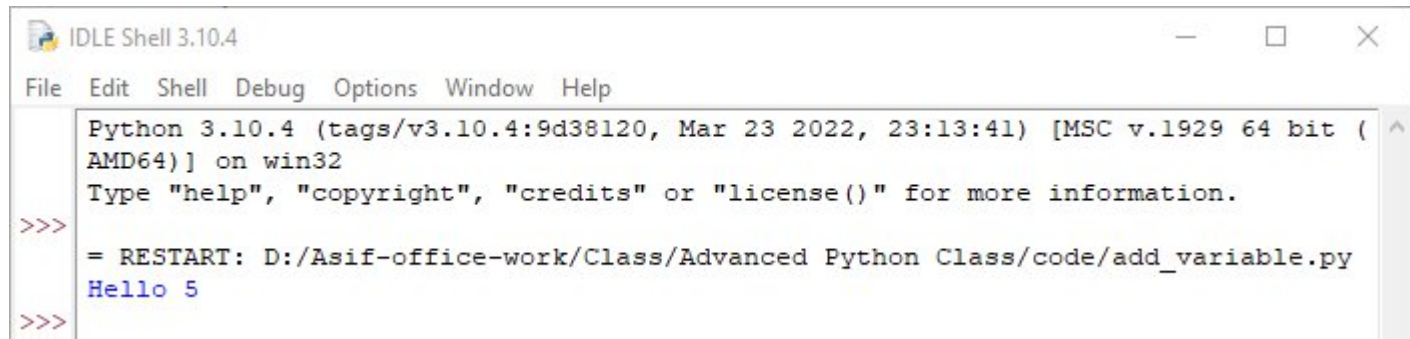
```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit  
AMD64] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
  
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/variable_sum.py  
32
```

Input:

A screenshot of a text editor window titled 'add_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/add_variable.py (3...'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code inside the editor is:

```
a = "Hello"  
b = 5  
print(a, b)
```

Output:

A screenshot of the IDLE Shell 3.10.4 window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The output text is:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/add_variable.py  
Hello 5  
>>>
```

Input:

```
a = "Hello"  
b = 5  
print(a + b)
```

Output:

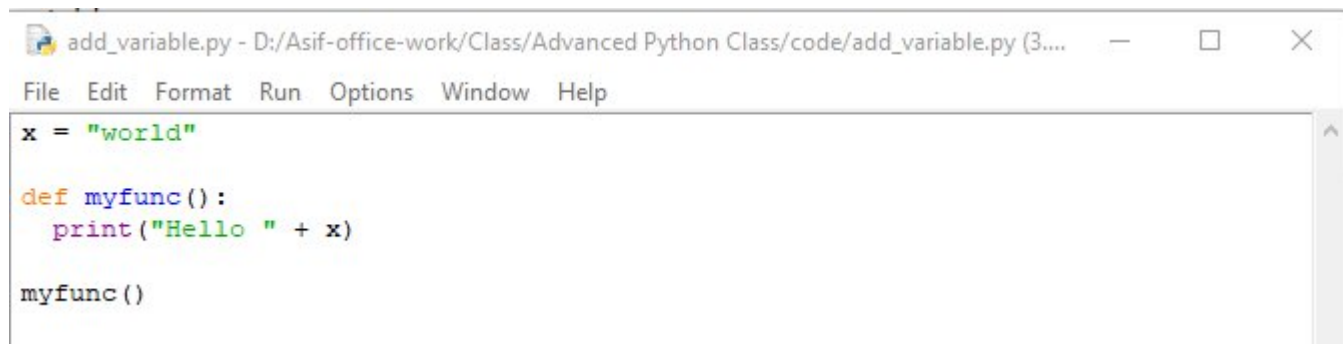
```
Traceback (most recent call last):  
  File "D:/Asif-office-work/Class/Advanced Python Class/code/variable_sum.py", line 3, in <module>  
    print(a + b)  
TypeError: can only concatenate str (not "int") to str
```

❑ **Global Variables**

Variables that are created outside of a function (as in all of the examples above) are known as global variables.

Global variables can be used by everyone, both inside of functions and outside.

Input:



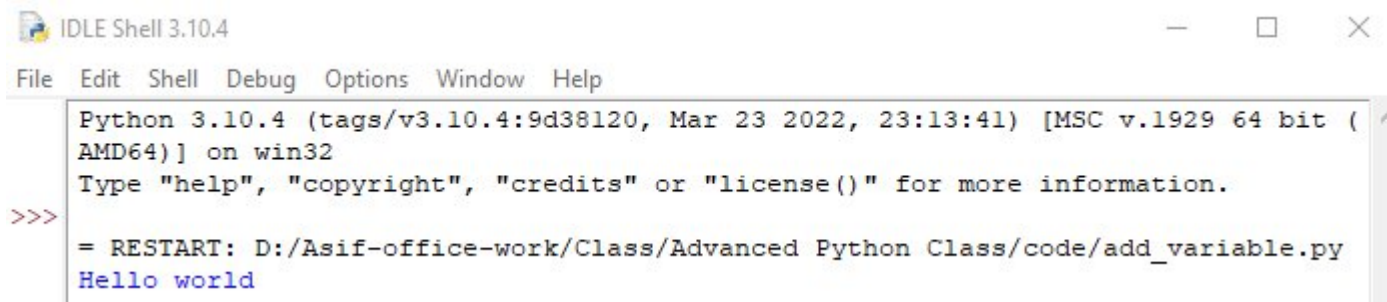
The screenshot shows a window titled 'add_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/add_variable.py (3....'. The menu bar includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor contains the following Python code:

```
x = "world"

def myfunc():
    print("Hello " + x)

myfunc()
```

Output:



The screenshot shows the 'IDLE Shell 3.10.4' window. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell displays the following output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/add_variable.py
Hello world
```

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.

Input:



```
global_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/global_variable...
File Edit Format Run Options Window Help

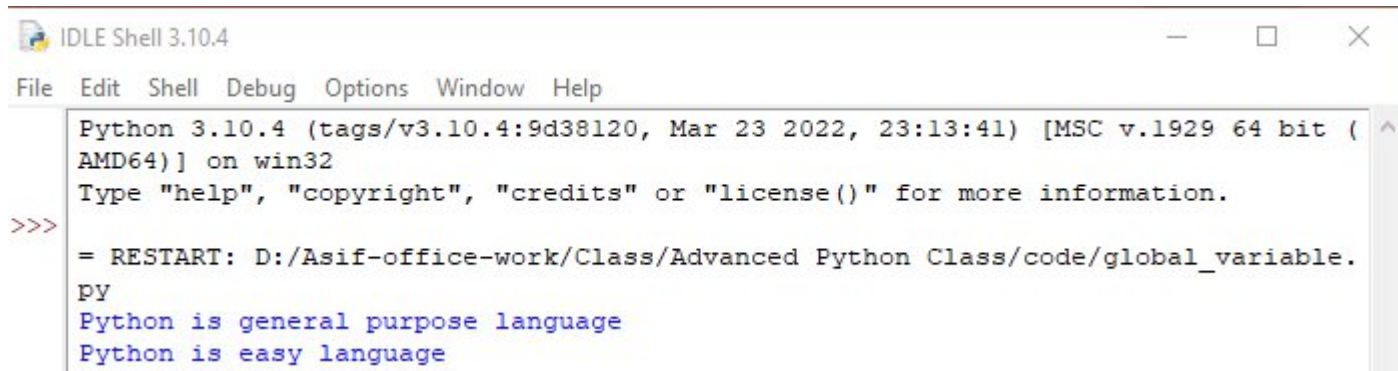
x = "easy"

def myfunc():
    x = "general purpose"
    print("Python is " + x+ " language")

myfunc()

print("Python is " + x+ " language")
```

Output:



```
IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help

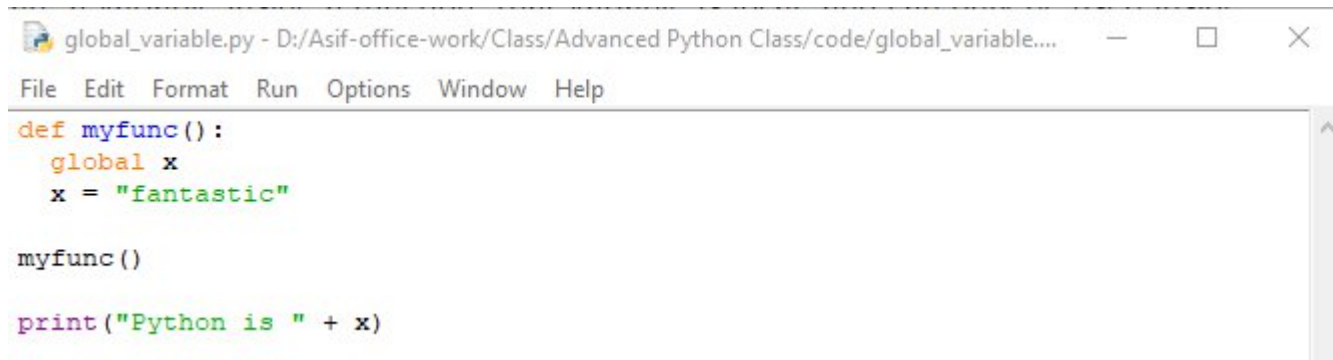
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/global_variable.PY
Python is general purpose language
Python is easy language
```

❑ The global Keyword

Normally, when you create a variable inside a function, that variable is local, and can only be used inside that function.

To create a global variable inside a function, you can use the **global** keyword.

Input:



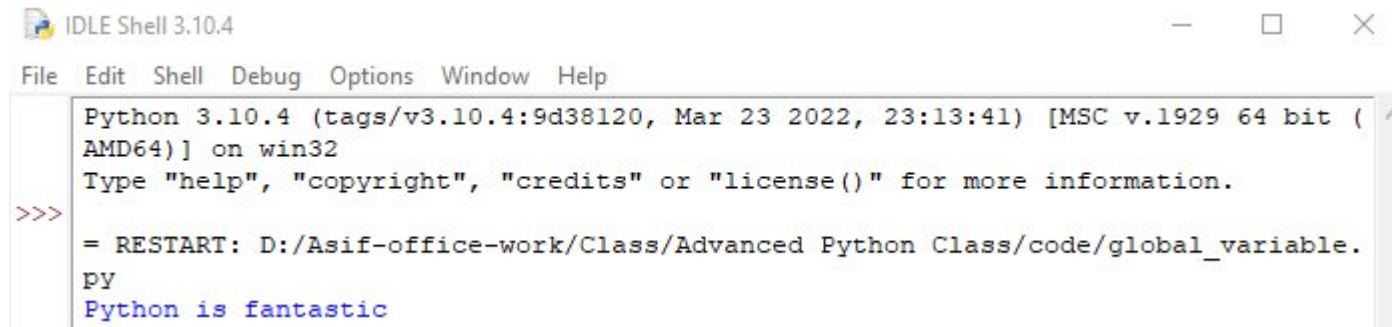
```
global_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/global_variable....
File Edit Format Run Options Window Help

def myfunc():
    global x
    x = "fantastic"

myfunc()

print("Python is " + x)
```

Output:

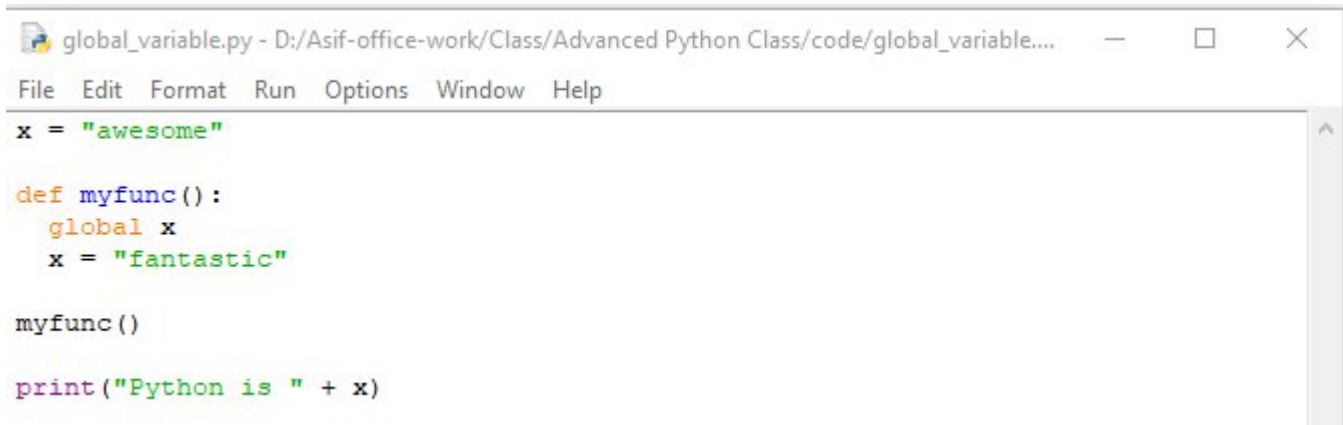


```
IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help

Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/global_variable.py
Python is fantastic
```

Also, use the **global** keyword if you want to change a global variable inside a function.

Input:



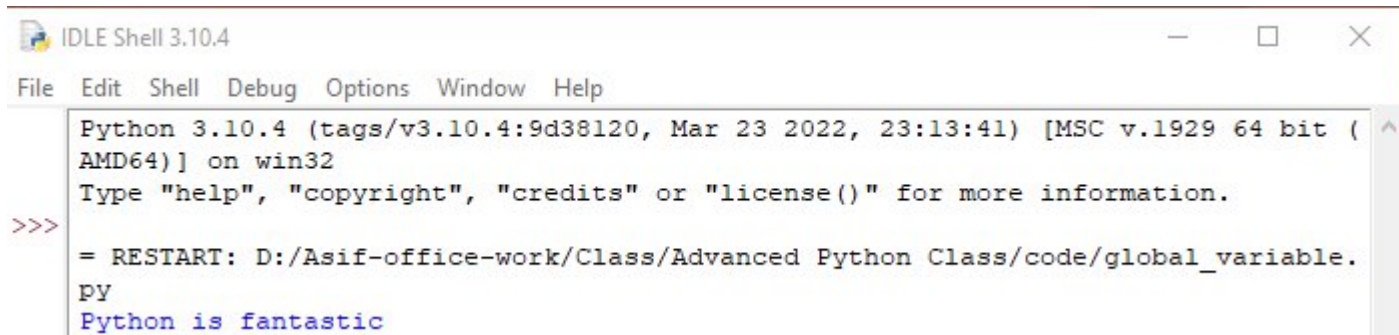
```
global_variable.py - D:/Asif-office-work/Class/Advanced Python Class/code/global_variable....
File Edit Format Run Options Window Help
x = "awesome"

def myfunc():
    global x
    x = "fantastic"

myfunc()

print("Python is " + x)
```

Output:



```
IDLE Shell 3.10.4
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/global_variable.PY
Python is fantastic
```

5. Data type:

❑ Built-in Data Types

Text Type: `str`

Numeric Types: `int`, `float`, `complex`

Sequence Types: `list`, `tuple`, `range`

Mapping Type: `dict`

Set Types: `set`, `frozenset`

Boolean Type: `bool`

Binary Types: `bytes`, `bytearray`, `memoryview`

❑ Numeric data type:

There are three numeric types in Python:

- Int
- Float
- Complex

```
a=10 #int  
b=3.8 #float  
c=12j #complex
```

❑ Check Type of a Variable:

Input:

```
x = 10  
print(type(x))
```

Output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
  
==== RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/type.py ====  
<class 'int'>
```

❑ Setting the Data Type

In Python, the data type is set when you assign a value to a variable:

Example	Data Type
<code>x = "Hello World"</code>	str
<code>x = 20</code>	int
<code>x = 20.5</code>	float
<code>x = 1j</code>	complex
<code>x = ["apple", "banana", "cherry"]</code>	list
<code>x = ("apple", "banana", "cherry")</code>	tuple
<code>x = range(6)</code>	range

Example	Data Type
<code>x = {"name": "John", "age": 36}</code>	dict
<code>x = {"apple", "banana", "cherry"}</code>	set
<code>x = frozenset({"apple", "banana", "cherry"})</code>	frozenset
<code>x = True</code>	bool
<code>x = b"Hello"</code>	bytes
<code>x = bytearray(5)</code>	bytearray
<code>x = memoryview(bytes(5))</code>	memoryview

❑ Setting the Specific Data Type

If you want to specify the data type, you can use the following constructor functions:

Example	Data Type
<code>x = str("Hello World")</code>	<code>str</code>
<code>x = int(20)</code>	<code>int</code>
<code>x = float(20.5)</code>	<code>float</code>
<code>x = complex(1j)</code>	<code>complex</code>
<code>x = list(("apple", "banana", "cherry"))</code>	<code>list</code>
<code>x = tuple(("apple", "banana", "cherry"))</code>	<code>tuple</code>

Example	Data Type
<code>x = range(6)</code>	<code>range</code>
<code>x = dict(name="John", age=36)</code>	<code>dict</code>
<code>x = set(("apple", "banana", "cherry"))</code>	<code>set</code>
<code>x = frozenset(("apple", "banana", "cherry"))</code>	<code>frozenset</code>
<code>x = bool(5)</code>	<code>bool</code>
<code>x = bytes(5)</code>	<code>bytes</code>
<code>x = bytearray(5)</code>	<code>bytearray</code>
<code>x = memoryview(bytes(5))</code>	<code>memoryview</code>

6.Type conversion:

❑ Python casting:

int:

```
a=int(5)      # a will be 5  
b=int(3.8)    # b will be 3  
c=int("12")   # c will be 12
```

Float:

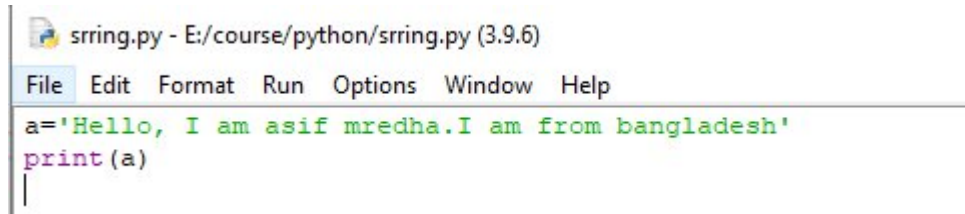
```
x=float(5)    # a will be 5.0  
y=float(3.8)  # b will be 3.8  
z=float("12") # c will be 12.0  
w=float("45.7") # c will be 45.7
```

String:

```
a=str("5") # a will be "5"  
b=str(3.8) # b will be "3.8"  
c=str("a1") # c will be "a1"  
d=str("T2") # c will be "T2"
```

7. Python String:

Input:



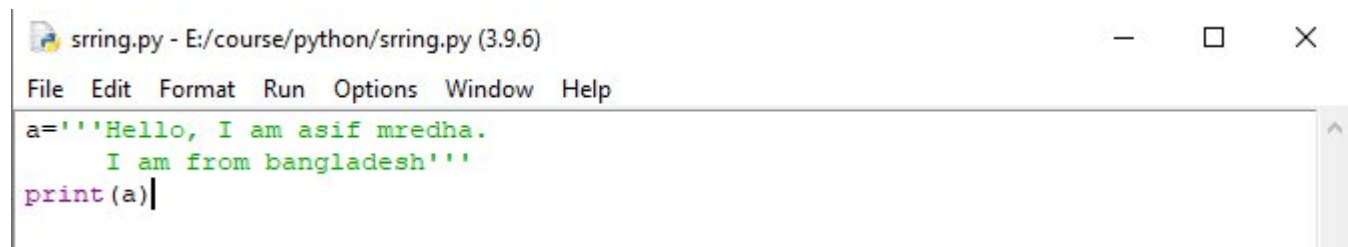
```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='Hello, I am asif mredha.I am from bangladesh'
print(a)
```

Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
Hello, I am asif mredha.I am from bangladesh
>>> |
```

Input:



```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='''Hello, I am asif mredha.
    I am from bangladesh'''
print(a)
```

Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
Hello, I am asif mredha.I am from bangladesh
>>>
===== RESTART: E:/course/python/srring.py =====
Hello, I am asif mredha.
    I am from bangladesh
>>> |
```

Input:

 type.py - D:/Asif-office-work/Class/Advanced Python Class/code/type.py (3.10.4)

File Edit Format Run Options Window Help

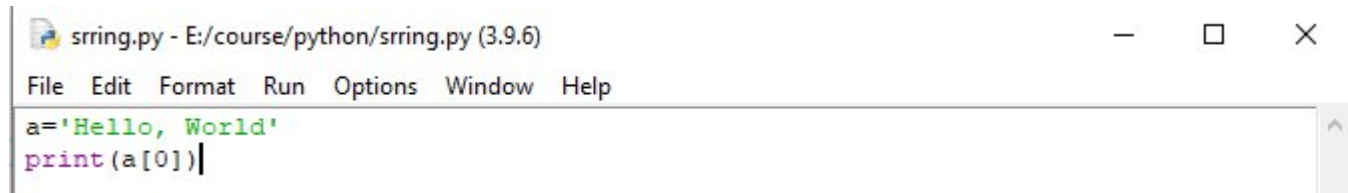
```
x="Hello,\n\tI am Md.Asif Mredha.\nI live in dhaka"  
print(x)
```

Output:

```
Hello,  
    I am Md.Asif Mredha.  
I live in dhaka
```

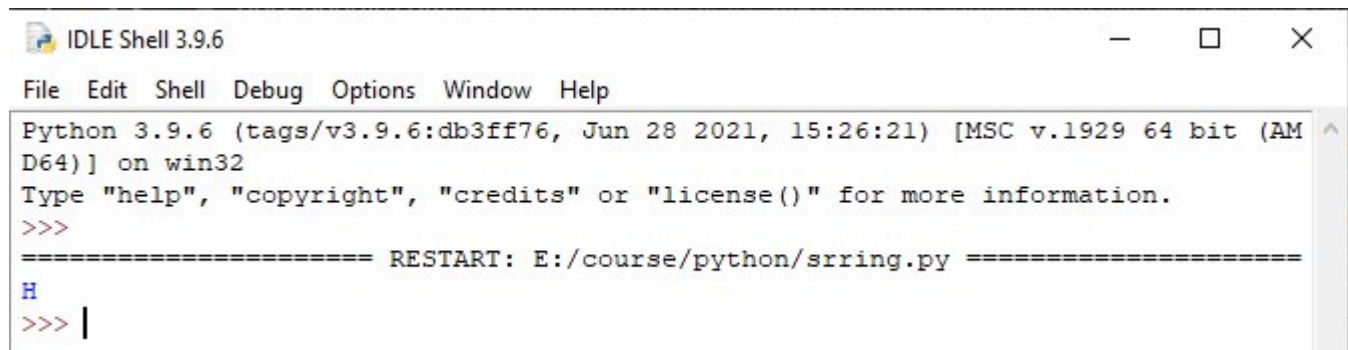
❑ Slicing String:

Input:

A screenshot of a Python script file named 'srring.py' located at 'E:/course/python/srring.py' using Python 3.9.6. The window has a standard title bar with minimize, maximize, and close buttons. The menu bar includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor contains two lines of Python code: `a='Hello, World'` and `print(a[0])`.

```
srring.py - E:/course/python/srring.py (3.9.6)
File Edit Format Run Options Window Help
a='Hello, World'
print(a[0])
```

Output:

A screenshot of the 'IDLE Shell 3.9.6' window. The title bar shows the window name and standard controls. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell displays the Python version and environment information, followed by a prompt. The script has been executed, resulting in the output 'H'. The prompt is now '>>> |'.

```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
H
>>> |
```

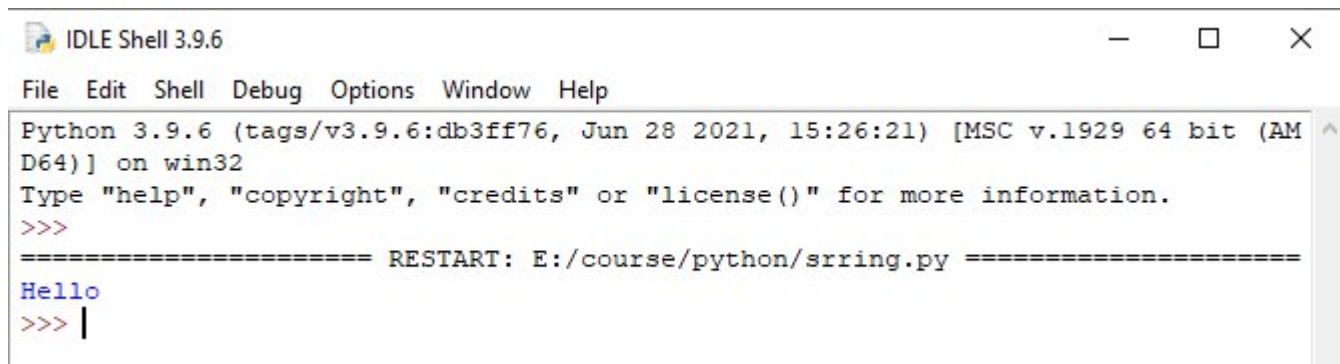
Input:



The screenshot shows a window titled "srring.py - E:/course/python/srring.py (3.9.6)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
a='Hello, World'
print(a[0:5])
```

Output:



The screenshot shows the "IDLE Shell 3.9.6" window. The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The shell displays the following text:

```
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/course/python/srring.py =====
Hello
>>> |
```

Input:

```
a='Hello world'
print(a[6:])
```

Output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

==== RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/type.py ====
world
```

Input:

```
a='Hello world'
print(a[-5:-2])
```

Output:

```
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:/Asif-office-work/Class/Advanced Python Class/code/type.py =====
```

```
wor
```

❑ Modify String:

➤ Lower:

Input:

```
File Edit Format Run Options Window Help
a = "Hello, World!"
print(a.upper())
```

Output:

```
File Edit Shell Debug Options Window Help
Python 3.10.4 (tags/v3.10.4:9d38120, Mar 23 2022, 23:13:41) [MSC v.1
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more informat
>>>
===== RESTART: D:/Asif-office-work/Batch/Python With Django/code/sl
HELLO, WORLD!
>>>
```

Input:

```
File Edit Format Run Options Window Help
a = "HELLO, WORLD!"
print(a.lower())
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python 1
hello, world!
```

➤ Remove **Whitespace**:

Input:

```
File Edit Format Run Options Window Help
a = " Hello, World! "
print(a.strip())
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python Wit
Hello, World!
```

➤ Replace:

Input:

```
File Edit Format Run Options Window Help
a = "Hello, Sijan!"
print(a.replace("S", "M"))
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python W.
Hello, Mijan!
```

➤ Split String:

Input:

```
File Edit Format Run Options Window Help
a = "Hello, World!"
print(a.split(","))
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python Wi
['Hello', ' World!']
```

➤ String Concatenation:

Input:

```
File Edit Format Run Options Window Help
a = "Hello"
b = "World"
c = a + " " + b
print(c)
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python
Hello World
```

➤ String Format:

Input:

```
File Edit Format Run Options Window Help
age = 12
text = "He is {} years old."
print(text.format(age))
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python 1
He is 12 years old.
```

Input:

```
File Edit Format Run Options Window Help
quantity = 3
item_no = 567
price = 49.95
myorder = "I want to buy {2} dollars for {0} pieces of item {1}."
print(myorder.format(quantity, item_no, price))
```

Output:

```
===== RESTART: D:/Asif-office-work/Batch/Python With Django/code.
I want to buy 49.95 dollars for 3 pieces of item 567.
```

➤ String Method

Method	Description
<u>capitalize()</u>	Converts the first character to upper case
<u>casefold()</u>	Converts string into lower case
<u>center()</u>	Returns a centered string
<u>count()</u>	Returns the number of times a specified value occurs in a string
<u>encode()</u>	Returns an encoded version of the string
<u>endswith()</u>	Returns true if the string ends with the specified value
<u>expandtabs()</u>	Sets the tab size of the string

Method	Description
<code>find()</code>	Searches the string for a specified value and returns the position of where it was found
<code>format()</code>	Formats specified values in a string
<code>format_map()</code>	Formats specified values in a string
<code>index()</code>	Searches the string for a specified value and returns the position of where it was found
<code>isalnum()</code>	Returns True if all characters in the string are alphanumeric
<code>isalpha()</code>	Returns True if all characters in the string are in the alphabet
<code>isdecimal()</code>	Returns True if all characters in the string are decimals

Method	Description
<u>isdigit()</u>	Returns True if all characters in the string are digits
<u>isidentifier()</u>	Returns True if the string is an identifier
<u>islower()</u>	Returns True if all characters in the string are lower case
<u>isnumeric()</u>	Returns True if all characters in the string are numeric
<u>isprintable()</u>	Returns True if all characters in the string are printable
<u>isspace()</u>	Returns True if all characters in the string are whitespaces
<u>isupper()</u>	Returns True if all characters in the string are upper case

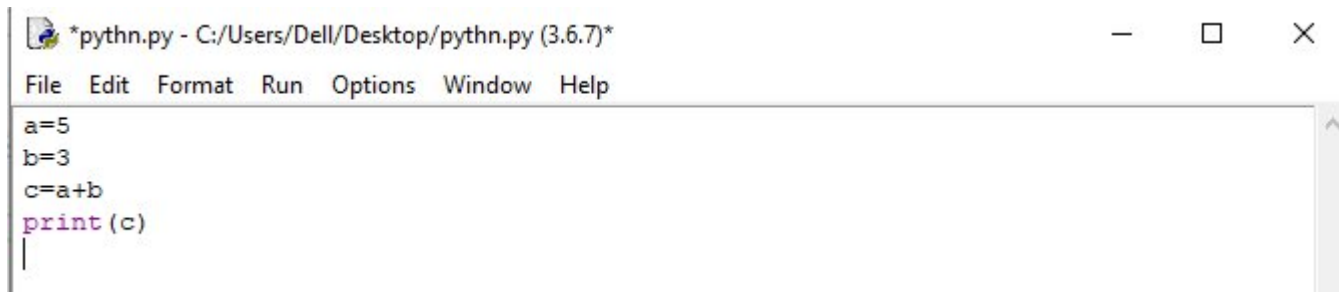
Method	Description
<u>join()</u>	Joins the elements of an iterable to the end of the string
<u>ljust()</u>	Returns a left justified version of the string
<u>lower()</u>	Converts a string into lower case
<u>lstrip()</u>	Returns a left trim version of the string
<u>maketrans()</u>	Returns a translation table to be used in translations
<u>partition()</u>	Returns a tuple where the string is parted into three parts
<u>replace()</u>	Returns a string where a specified value is replaced with a specified value

Method	Description
<u>rfind()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rindex()</u>	Searches the string for a specified value and returns the last position of where it was found
<u>rjust()</u>	Returns a right justified version of the string
<u>rpartition()</u>	Returns a tuple where the string is parted into three parts
<u>rsplit()</u>	Splits the string at the specified separator, and returns a list
<u>rstrip()</u>	Returns a right trim version of the string
<u>split()</u>	Splits the string at the specified separator, and returns a list

Method	Description
<u>splitlines()</u>	Splits the string at line breaks and returns a list
<u>startswith()</u>	Returns true if the string starts with the specified value
<u>strip()</u>	Returns a trimmed version of the string
<u>swapcase()</u>	Swaps cases, lower case becomes upper case and vice versa
<u>title()</u>	Converts the first character of each word to upper case
<u>translate()</u>	Returns a translated string
<u>upper()</u>	Converts a string into upper case
<u>zfill()</u>	Fills the string with a specified number of 0 values at the beginning

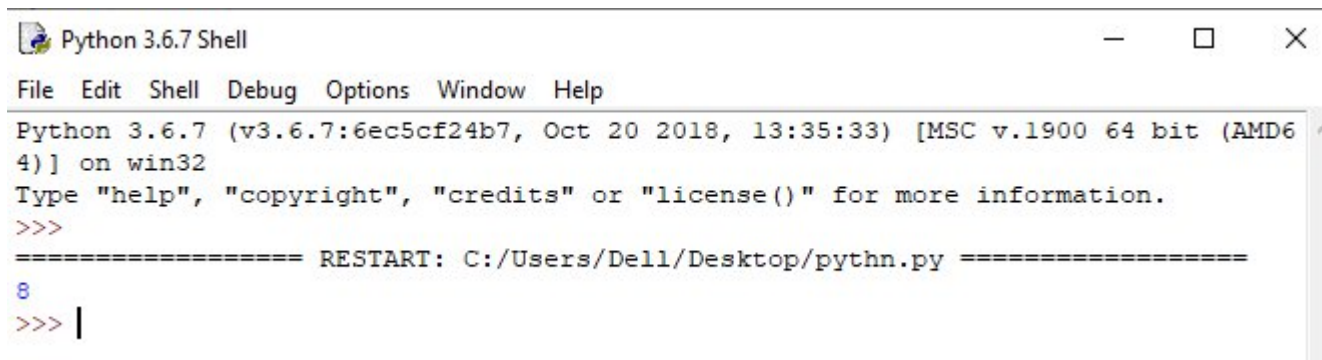
8. Print Function:

Input:



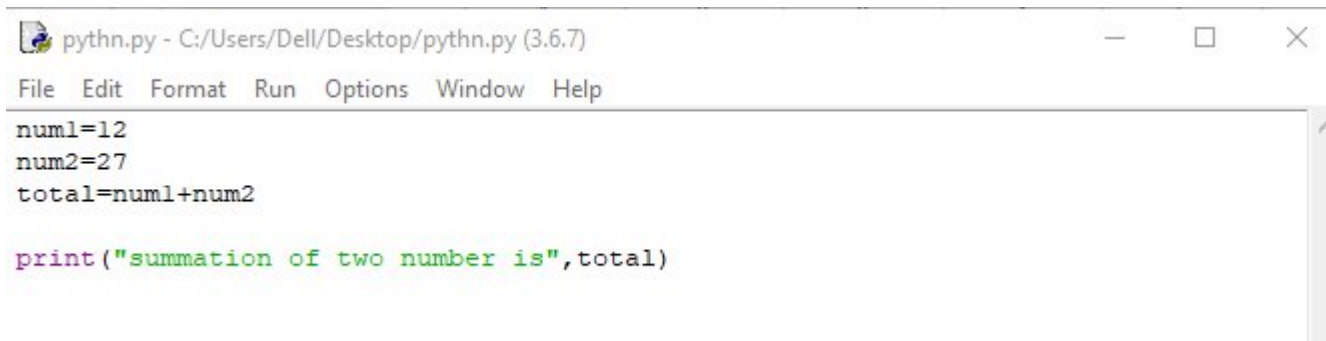
```
*pythn.py - C:/Users/Dell/Desktop/pythn.py (3.6.7)*
File Edit Format Run Options Window Help
a=5
b=3
c=a+b
print(c)
|
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/pythn.py =====
8
>>> |
```

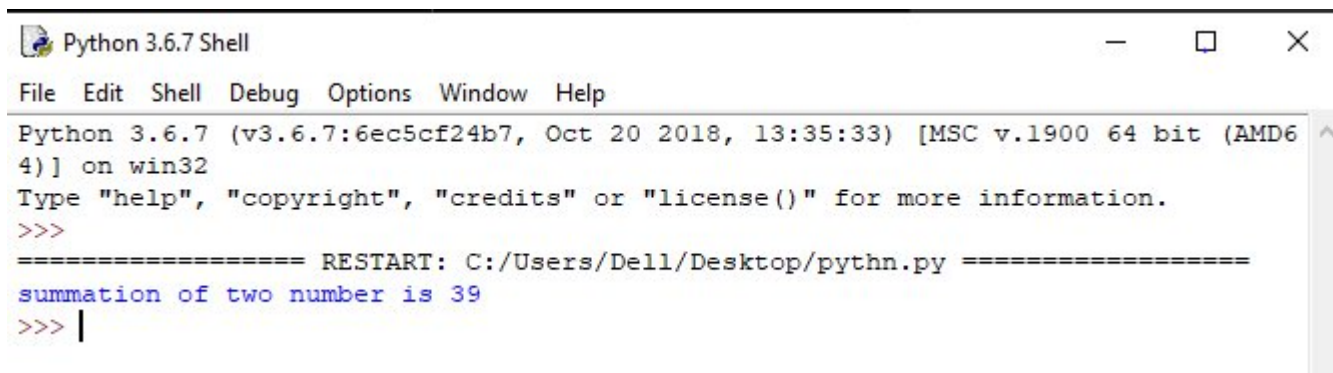
Input:



```
pythn.py - C:/Users/Dell/Desktop/pythn.py (3.6.7)
File Edit Format Run Options Window Help
num1=12
num2=27
total=num1+num2

print("summation of two number is",total)
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/pythn.py =====
summation of two number is 39
>>> |
```

Input:

```
num1=12
num2=27
total=num1+num2

print("summation of",num1,"and",num2,"is",total)
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 b
AMD64] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:/asif-office-work/python_code/pythn.py =====
summation of 12 and 27 is 39
|
```

❑ Format Method

Input:

```
File Edit Format Run Options Window Help
print("Hello {}, your balance is {}".format("Hridoy", 10035))
```

Output:

```
== RESTART: D:/Asif-office-work/Batch/Python With Django.
Hello Hridoy, your balance is 10035.
```

Input:

```
File Edit Format Run Options Window Help
num1=320
num2=130
total=num1+num2

print("Summation of {} and {} is {}".format(num1,num2,total))
```

Output:

```
== RESTART: D:/Asif-office-work/Batch/Pythor
Summation of 320 and 130 is 450
```

❑ F String:

Input:

```
File Edit Format Run Options Window Help
name="hasan"
balance=3500
print(f"Hello, {name}. You have {balance} Bdt.")
```

Output:

```
== RESTART: D:/Asif-office-work/Batch/Python With Dj
Hello, hasan. You have 3500 Bdt.
```

Input:

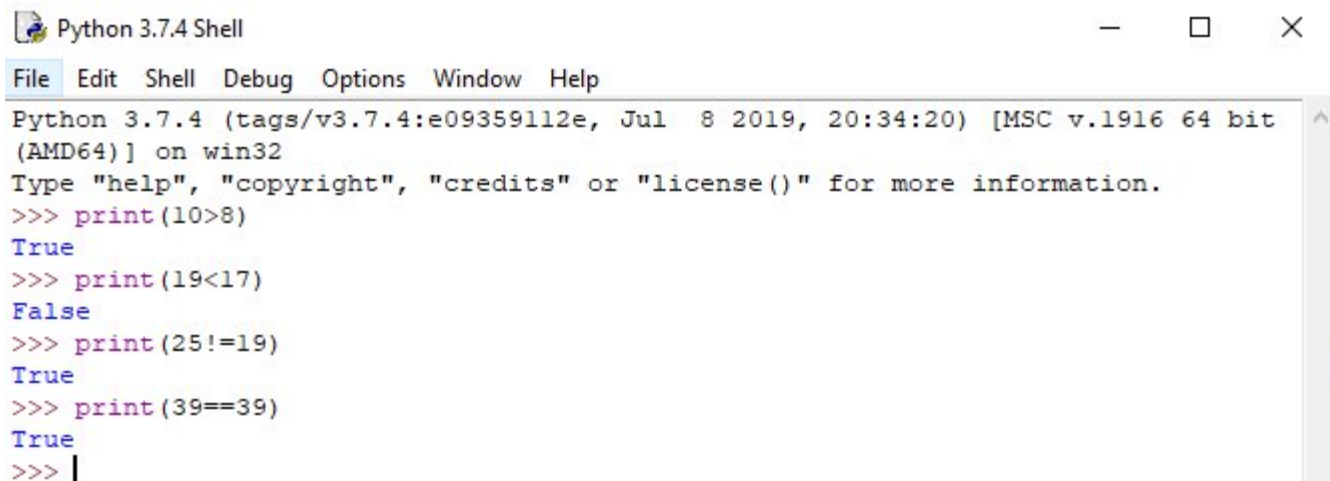
```
File Edit Format Run Options Window Help
num1=320
num2=130
total=num1+num2

print(f"Summation of {num1} and {num2} is {total}")
```

Output:

```
== RESTART: D:/Asif-office-work/Batch/Python With Django/pytho
Summation of 320 and 130 is 450
```

9. Python booleans:



```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print(10>8)
True
>>> print(19<17)
False
>>> print(25!=19)
True
>>> print(39==39)
True
>>> |
```

Input:

```
print(bool("Hello"))  
print(bool(15))
```

Output:

```
True  
True
```

Input:

```
x = "Hello"  
y = 21  
  
print(bool(x))  
print(bool(y))
```

Output:

```
True  
True
```

Input:

```
print(bool("abc"))  
print(bool(123))  
print(bool(["apple", "cherry", "banana"]))
```

Output:

```
True  
True  
True
```

Input:

```
print(bool(False))  
print(bool(None))  
print(bool(0))  
print(bool(""))  
print(bool(()))  
print(bool([]))  
print(bool({}))
```

Output:

```
False  
False  
False  
False  
False  
False  
False
```

10. Python Operator:

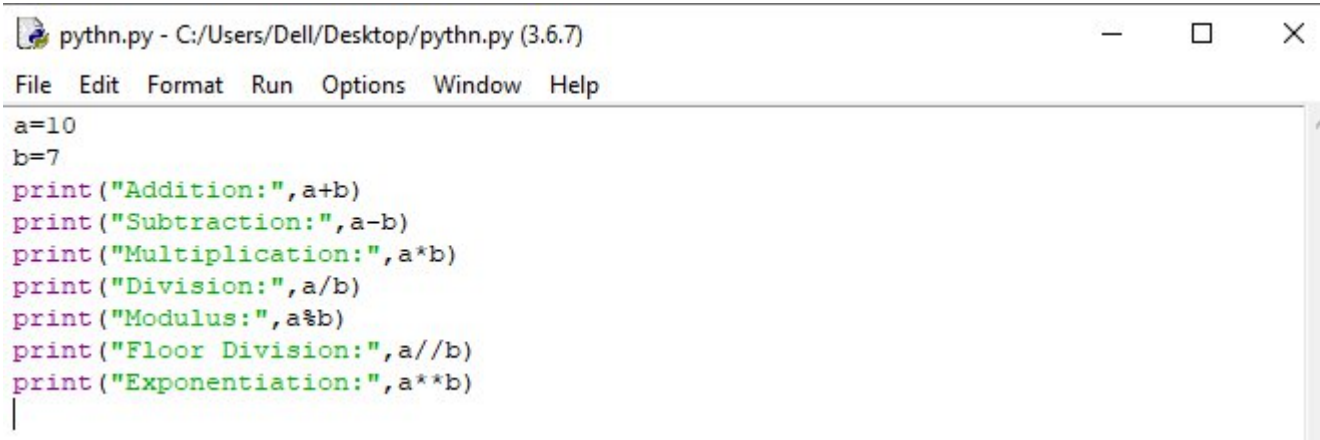
Operators are used to perform operations on variables.

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Identity Operators
- Membership Operators
- Bitwise Operators

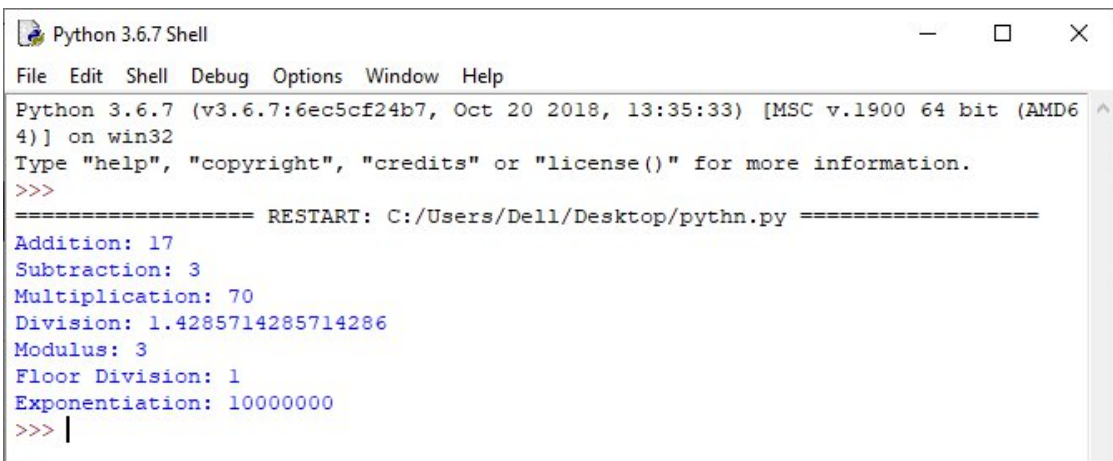
❑ Arithmetic Operator:

Input:

Output:



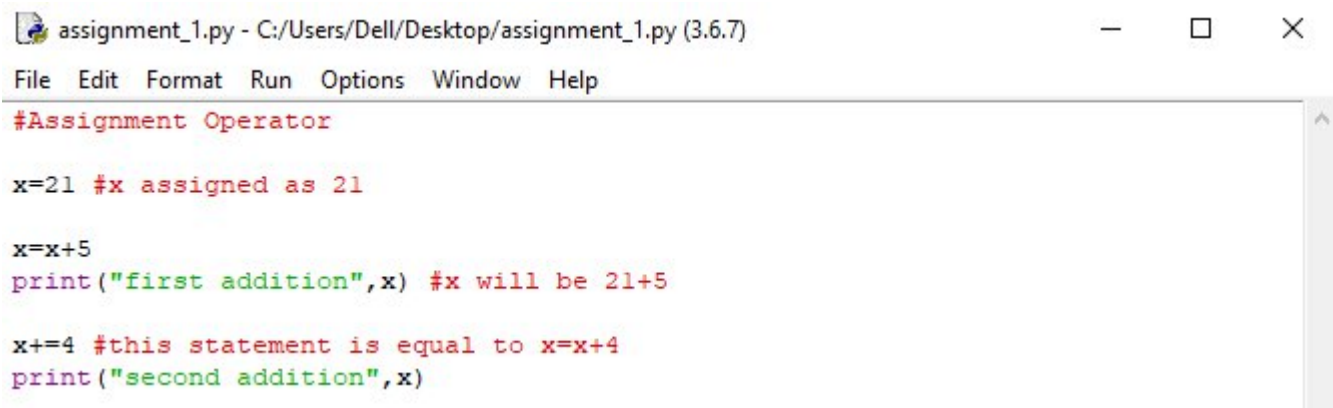
```
pythn.py - C:/Users/Dell/Desktop/pythn.py (3.6.7)
File Edit Format Run Options Window Help
a=10
b=7
print("Addition:",a+b)
print("Subtraction:",a-b)
print("Multiplication:",a*b)
print("Division:",a/b)
print("Modulus:",a%b)
print("Floor Division:",a//b)
print("Exponentiation:",a**b)
```



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/pythn.py =====
Addition: 17
Subtraction: 3
Multiplication: 70
Division: 1.4285714285714286
Modulus: 3
Floor Division: 1
Exponentiation: 10000000
>>>
```

❑ Assignment Operator:

Input:



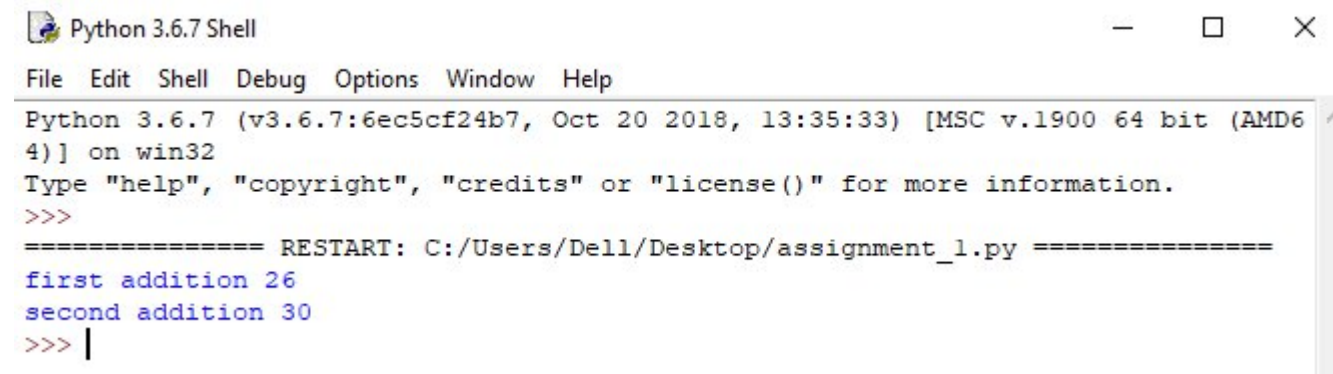
```
assignment_1.py - C:/Users/Dell/Desktop/assignment_1.py (3.6.7)
File Edit Format Run Options Window Help
#Assignment Operator

x=21 #x assigned as 21

x=x+5
print("first addition",x) #x will be 21+5

x+=4 #this statement is equal to x=x+4
print("second addition",x)
```

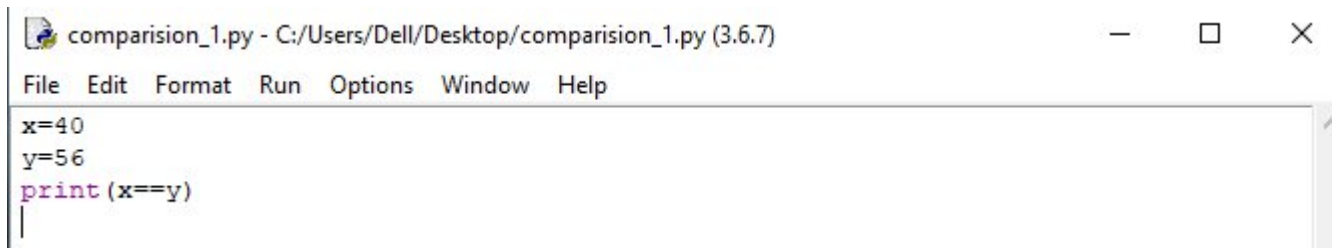
Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/assignment_1.py =====
first addition 26
second addition 30
>>> |
```

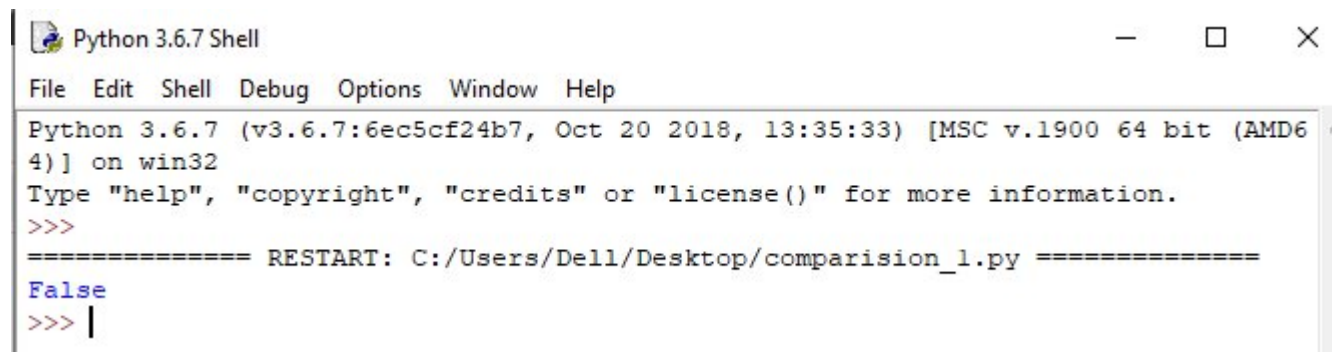
❑ Comparison Operator:

Input:



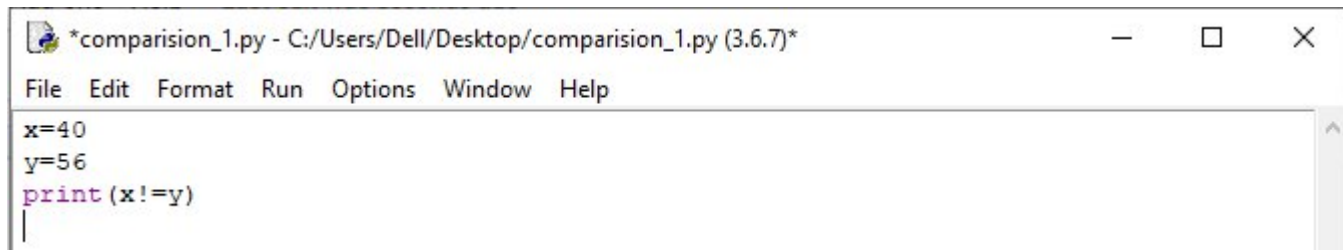
```
comparision_1.py - C:/Users/Dell/Desktop/comparision_1.py (3.6.7)
File Edit Format Run Options Window Help
x=40
y=56
print(x==y)
|
```

Output:



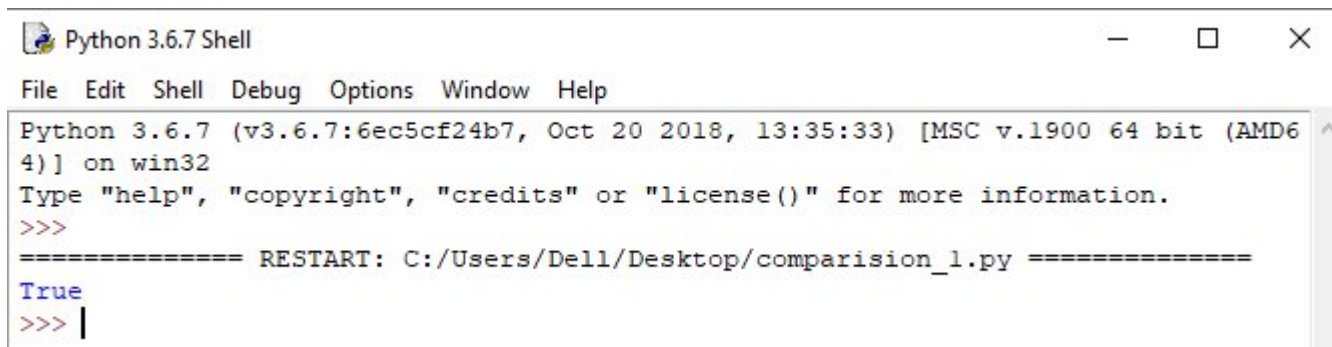
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/comparision_1.py =====
False
>>> |
```

Input:



```
*comparision_1.py - C:/Users/Dell/Desktop/comparision_1.py (3.6.7)*
File Edit Format Run Options Window Help
x=40
y=56
print(x!=y)
|
```

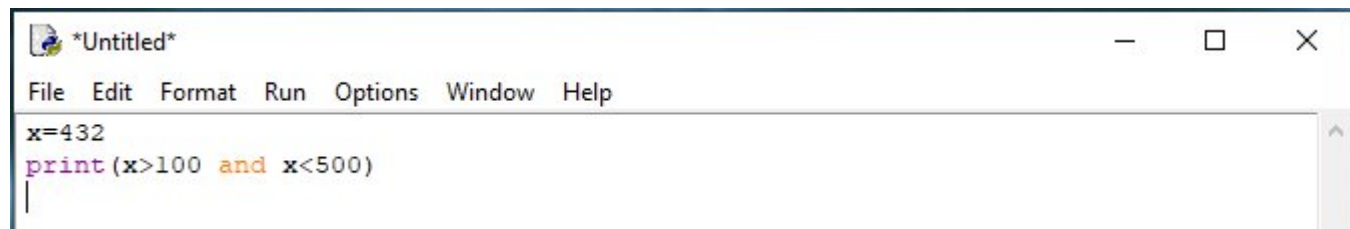
Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/comparision_1.py =====
True
>>> |
```

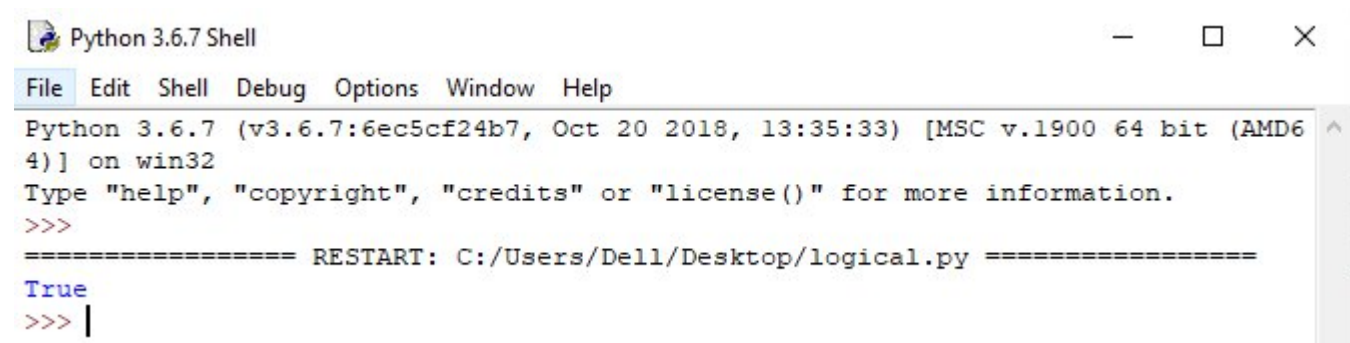
❑ Logical Operator:

Input:



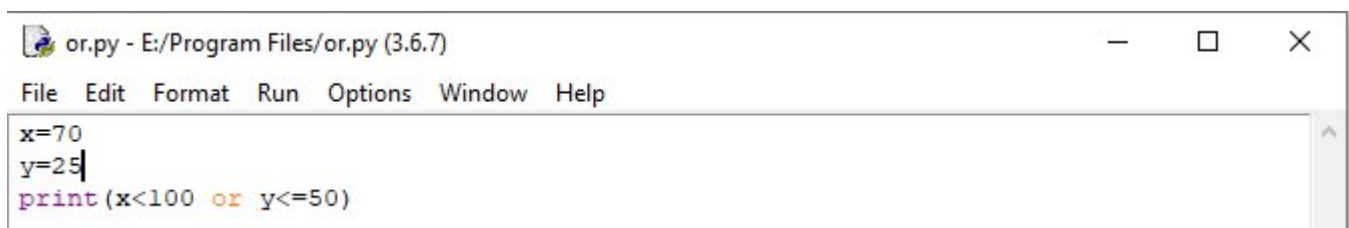
```
*Untitled*
File Edit Format Run Options Window Help
x=432
print(x>100 and x<500)
|
```

Output:



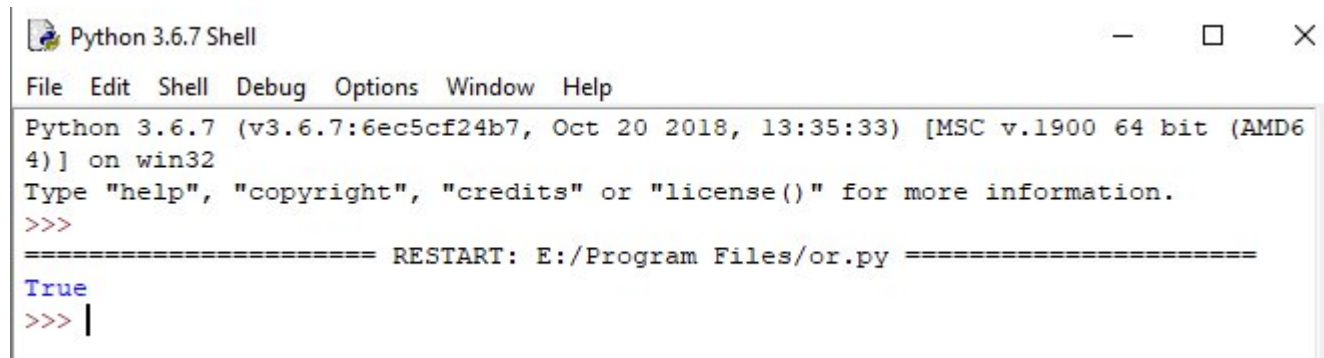
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/logical.py =====
True
>>> |
```

Input:



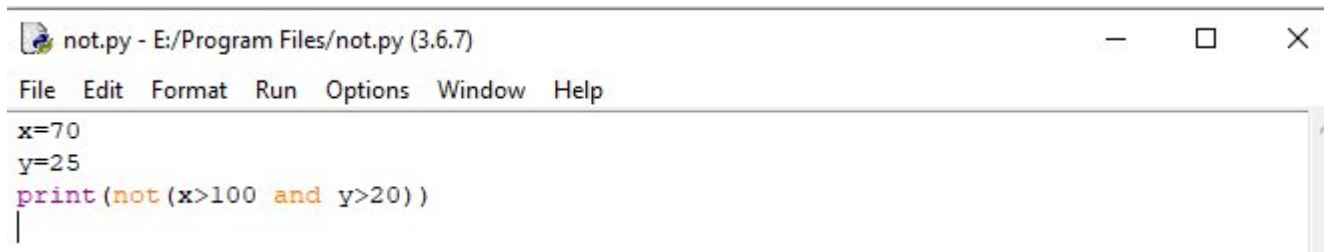
```
or.py - E:/Program Files/or.py (3.6.7)
File Edit Format Run Options Window Help
x=70
y=25
print(x<100 or y<=50)
```

Output:



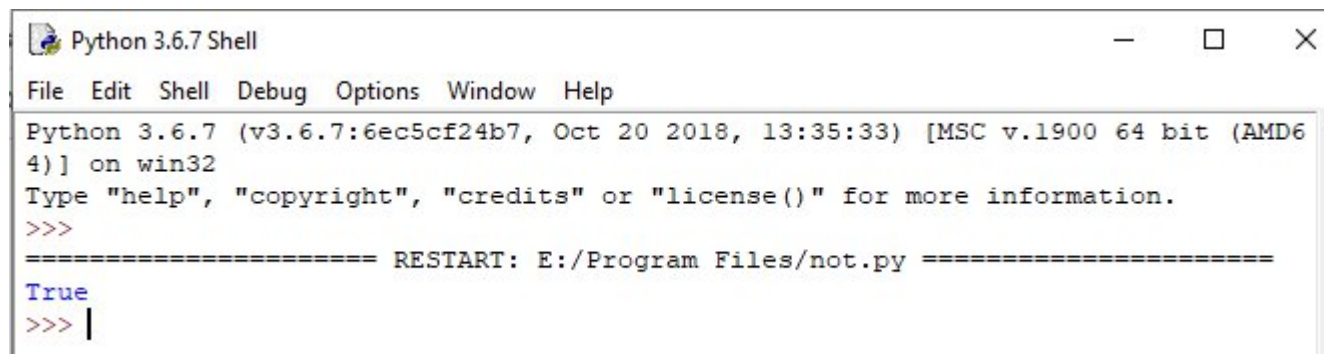
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/Program Files/or.py =====
True
>>> |
```

Input:



```
not.py - E:/Program Files/not.py (3.6.7)
File Edit Format Run Options Window Help
x=70
y=25
print(not(x>100 and y>20))
|
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/Program Files/not.py =====
True
>>> |
```

❑ Identity Operators

Input:

```
x = ["apple", "mango"]
```

```
y = ["apple", "mango"]
```

```
a = x
```

```
print(x is a)
```

```
# returns True because a is the same object as x
```

```
print(x is y)
```

```
# returns False because x is not the same object as y, even if they have the same content
```

```
print(x == y)
```

```
# to demonstrate the difference between "is" and "==": this comparison returns True because x is equal to y
```

Output:

```
True  
False  
True
```

Input:

```
x = ["apple", "mango"]
y = ["apple", "mango"]

a = x

print(x is not a)

# returns False because z is the same object as x

print(x is not y)

# returns True because x is not the same object as y, even if they have the same content

print(x != y)

# to demonstrate the difference between "is not" and "!=": this comparison returns False because x is equal to y
```

Output:

```
False
True
False
```

❑ Python Membership Operators:

Input:

```
x = ["apple", "mango"]  
  
print("mango" in x)  
  
# returns True because a sequence with the value "mango" is in the list
```

Output:

```
True
```

Input:

```
x = ["apple", "mango"]  
  
print("pineapple" not in x)  
  
# returns True because a sequence with the value "pineapple" is not in  
the list
```

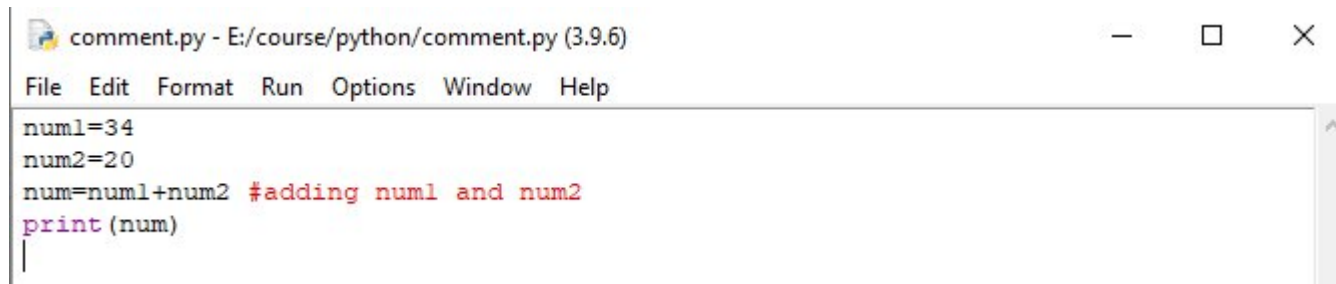
Output:

```
True
```

11. Comments:

- A Python comment is a line of text in a program that is not executed by the interpreter. Comments are used during debugging to identify issues and to explain code.
- A Single line comment in Python starts with the hash character # and extends to the end of the physical line.
- A Multiline comment in Python can be created using triple-quoted strings

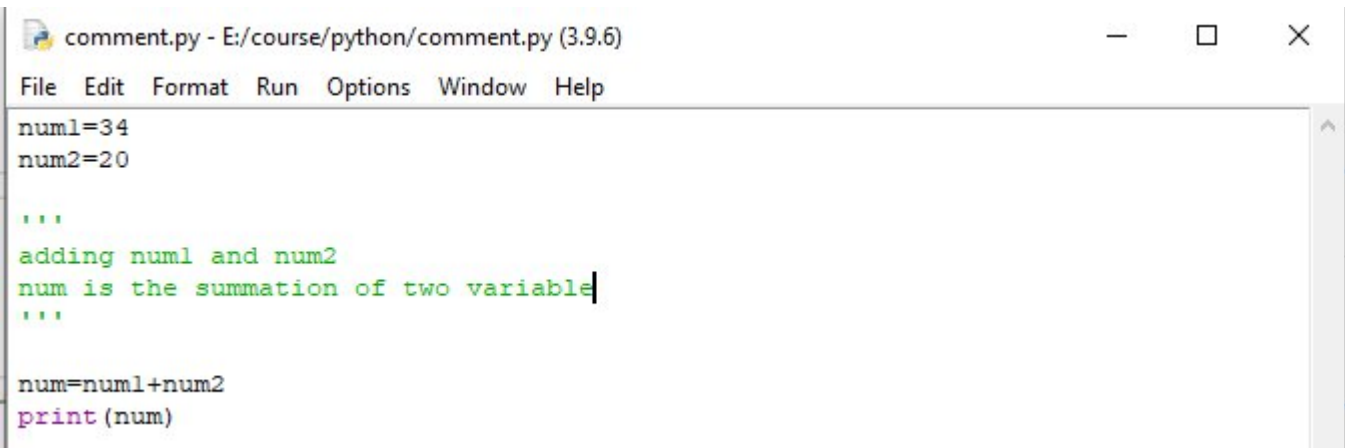
❑ Comment Using



The screenshot shows a window titled "comment.py - E:/course/python/comment.py (3.9.6)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

```
num1=34
num2=20
num=num1+num2 #adding num1 and num2
print(num)
```

❑ Comment using triple-quotation



The screenshot shows a window titled "comment.py - E:/course/python/comment.py (3.9.6)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

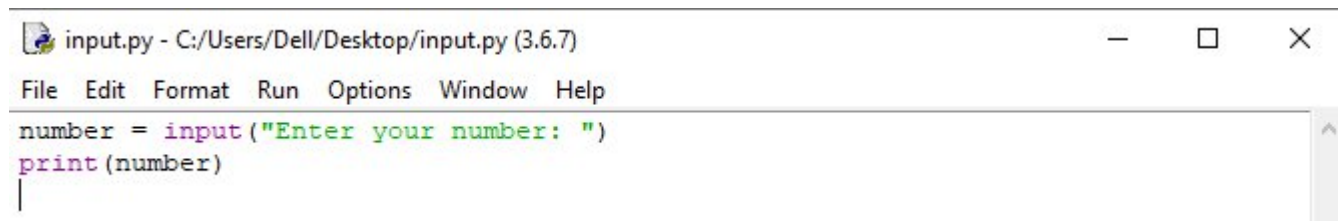
```
num1=34
num2=20

'''
adding num1 and num2
num is the summation of two variable
'''

num=num1+num2
print(num)
```

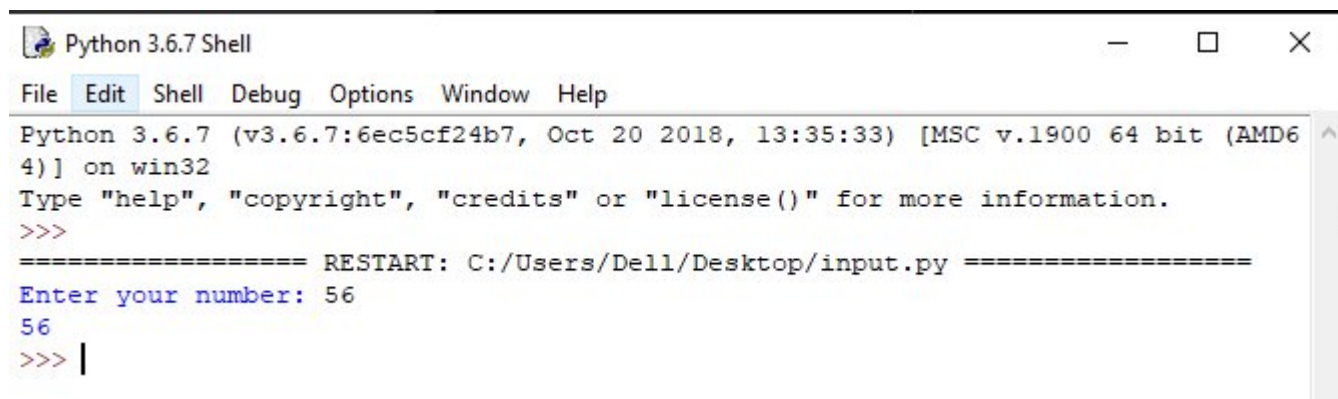
12.Input Method:

Input:



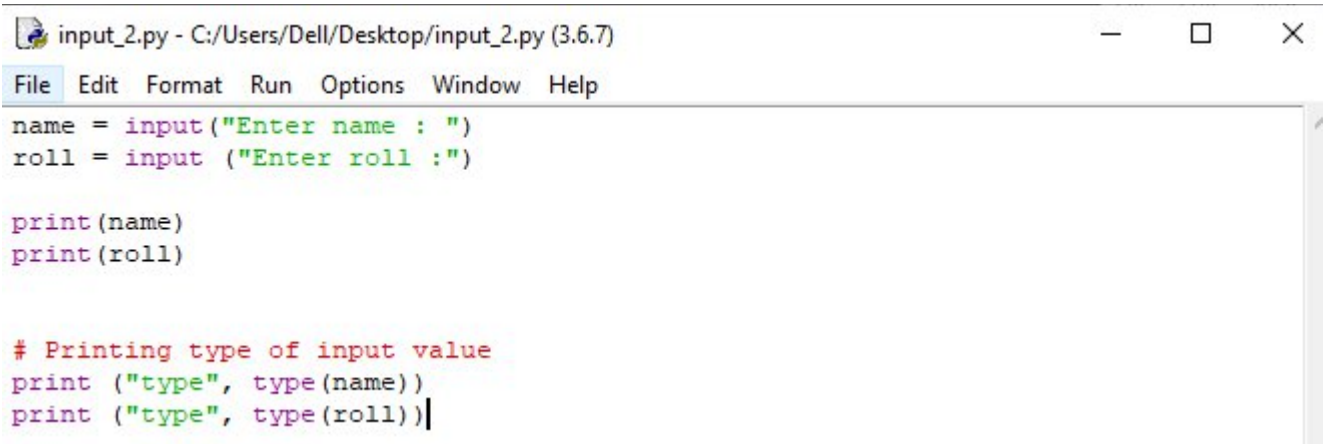
```
input.py - C:/Users/Dell/Desktop/input.py (3.6.7)
File Edit Format Run Options Window Help
number = input("Enter your number: ")
print(number)
```

Output:



```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/input.py =====
Enter your number: 56
56
>>> |
```

Input:

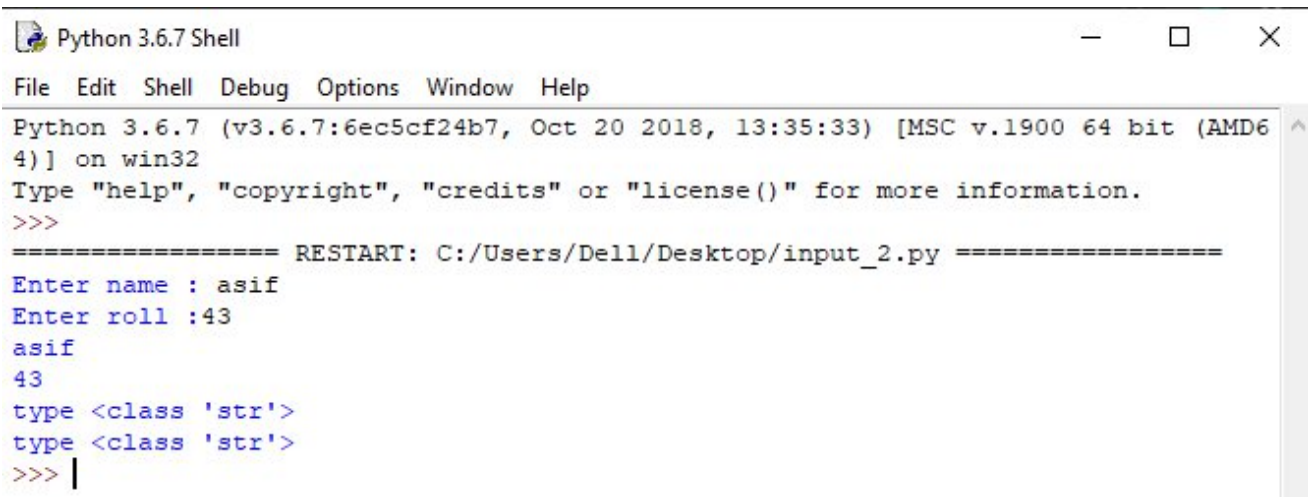


```
input_2.py - C:/Users/Dell/Desktop/input_2.py (3.6.7)
File Edit Format Run Options Window Help
name = input("Enter name : ")
roll = input("Enter roll :")

print(name)
print(roll)

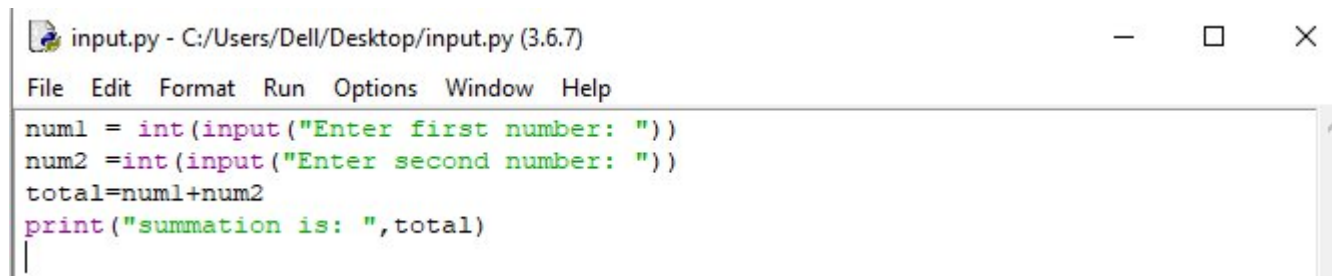
# Printing type of input value
print("type", type(name))
print("type", type(roll))
```

Output:



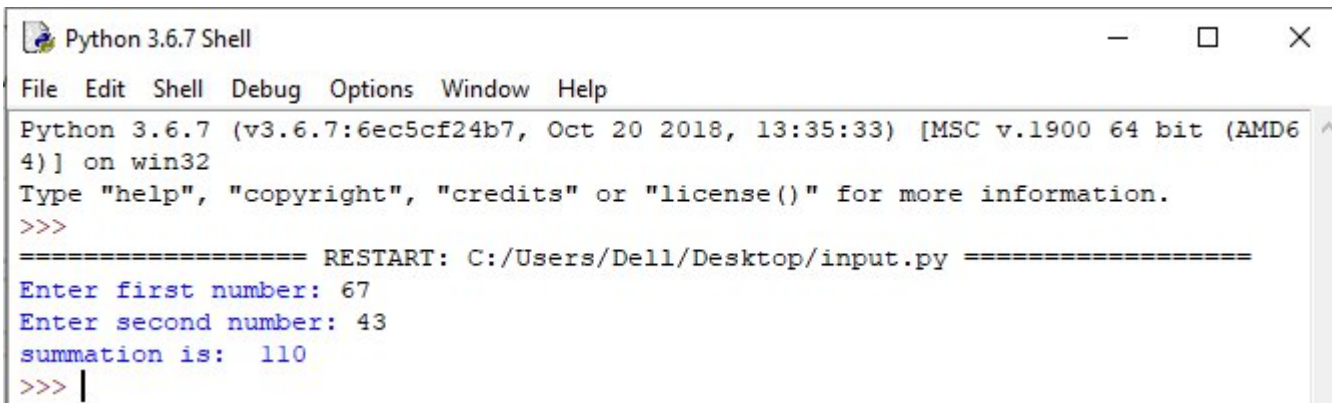
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/input_2.py =====
Enter name : asif
Enter roll :43
asif
43
type <class 'str'>
type <class 'str'>
>>> |
```

Input:



```
input.py - C:/Users/Dell/Desktop/input.py (3.6.7)
File Edit Format Run Options Window Help
num1 = int(input("Enter first number: "))
num2 =int(input("Enter second number: "))
total=num1+num2
print("summation is: ",total)
|
```

Output:



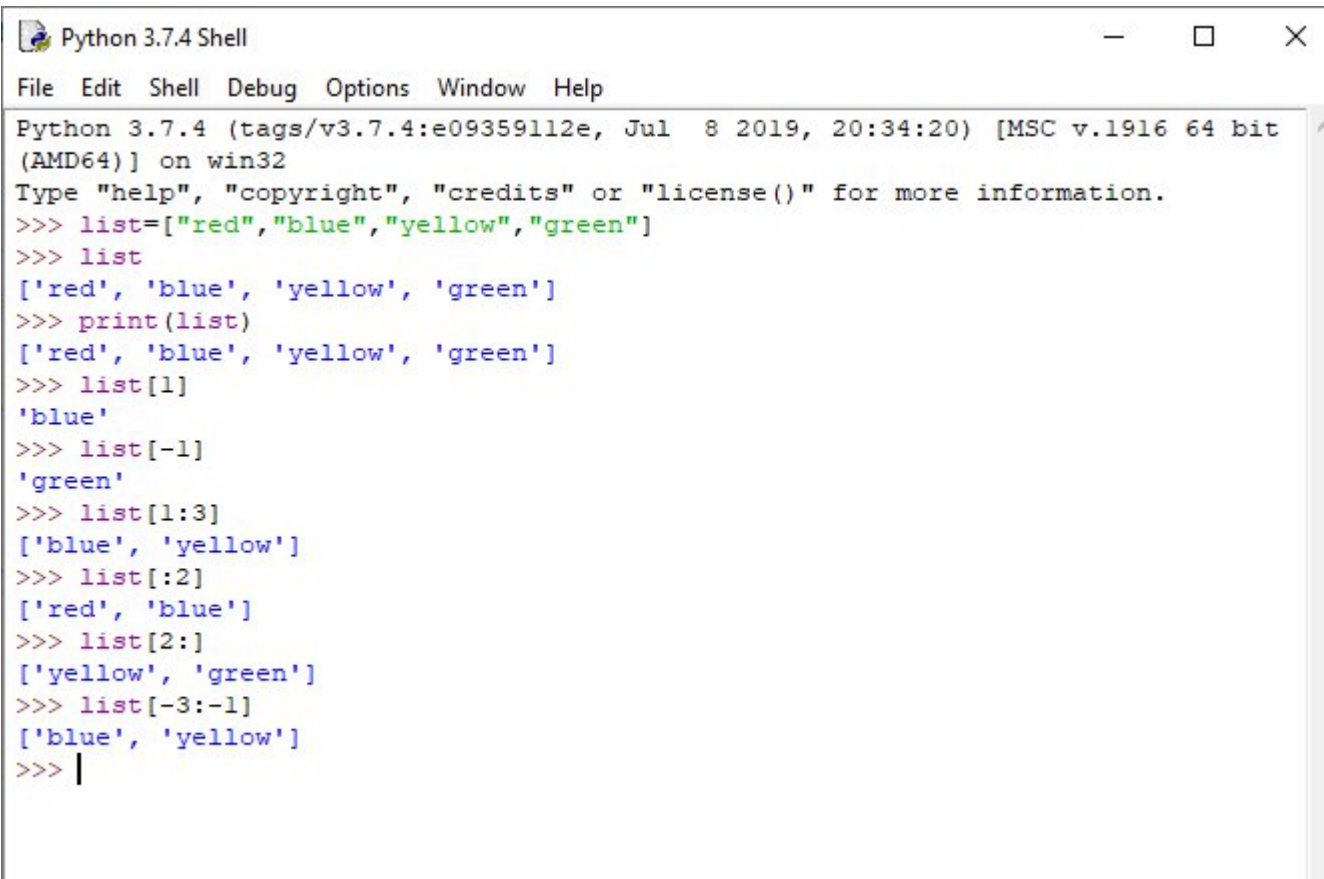
```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/input.py =====
Enter first number: 67
Enter second number: 43
summation is: 110
>>> |
```

13.Python Collection:

There are four collection data types in the Python programming language:

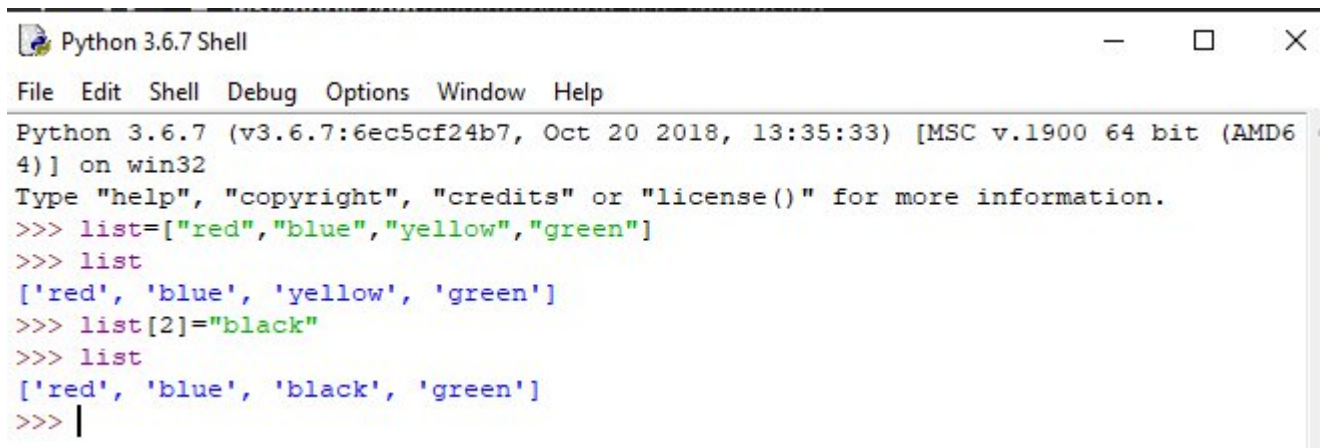
- 1.List** is a collection which is ordered and changeable. Allows duplicate members.
- 2.Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- 3.Set** is a collection which is unordered, unchangeable, and unindexed. No duplicate members.
- 4.Dictionary** is a collection which is ordered and changeable. No duplicate members.

□ List:

A screenshot of a Python 3.7.4 Shell window. The window has a title bar with the text "Python 3.7.4 Shell" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main area of the window contains a text-based interface for the Python shell. It shows the version and build information: "Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)] on win32". It also displays a prompt to type "help", "copyright", "credits" or "license()" for more information. The user has entered several commands to create and manipulate a list. The list is created with the elements "red", "blue", "yellow", and "green". The user then prints the list, accesses individual elements by index, and uses slicing to create sublists. The output of each command is displayed on the line immediately following the command.

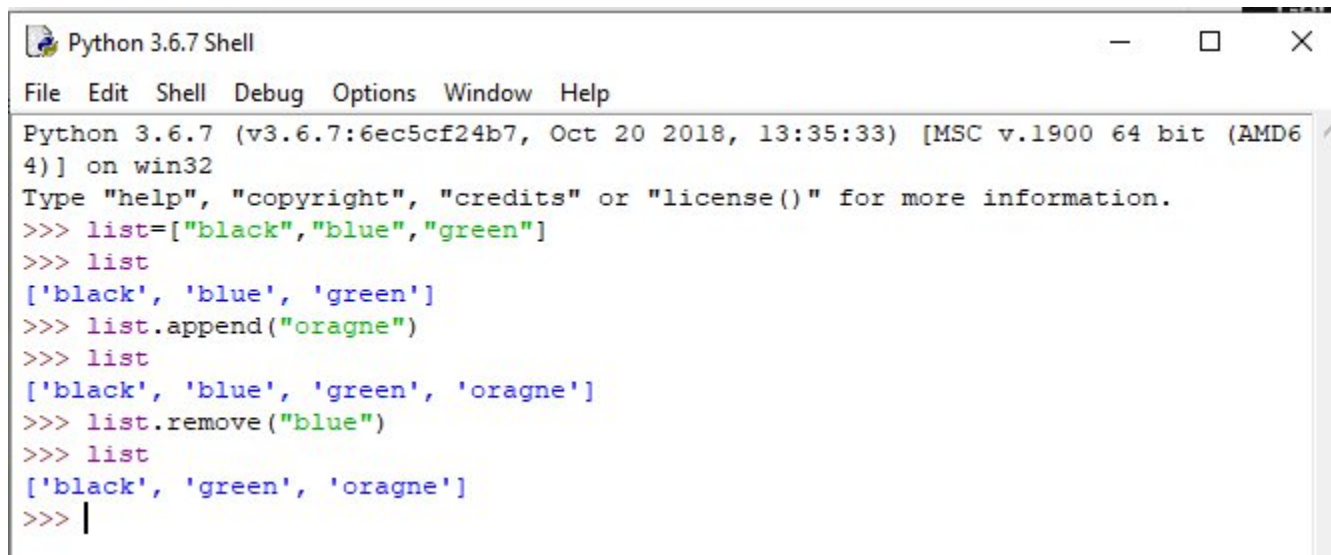
```
Python 3.7.4 Shell
File Edit Shell Debug Options Window Help
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list=["red","blue","yellow","green"]
>>> list
['red', 'blue', 'yellow', 'green']
>>> print(list)
['red', 'blue', 'yellow', 'green']
>>> list[1]
'blue'
>>> list[-1]
'green'
>>> list[1:3]
['blue', 'yellow']
>>> list[:2]
['red', 'blue']
>>> list[2:]
['yellow', 'green']
>>> list[-3:-1]
['blue', 'yellow']
>>> |
```

➤ Change Item Value:

A screenshot of a Python 3.6.7 Shell window. The window has a title bar with the text "Python 3.6.7 Shell" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main area of the window displays the following text:

```
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list=["red","blue","yellow","green"]
>>> list
['red', 'blue', 'yellow', 'green']
>>> list[2]="black"
>>> list
['red', 'blue', 'black', 'green']
>>> |
```

➤ **Add and remove value from List:**

A screenshot of a Python 3.6.7 Shell window. The window has a title bar with the text "Python 3.6.7 Shell" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with the items "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main area of the window contains the following text:

```
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list=["black","blue","green"]
>>> list
['black', 'blue', 'green']
>>> list.append("orange")
>>> list
['black', 'blue', 'green', 'orange']
>>> list.remove("blue")
>>> list
['black', 'green', 'orange']
>>> |
```

➤ List Comprehension:

Input:

```
fruits = ["apple", "mango", "cherry", "banana", "pineapple"]
new_list = []

for x in fruits:
    if "p" in x:
        new_list.append(x)

print(new_list)
```

Output:

```
['apple', 'pineapple']
```

➤ Sort Lists:

Input:

```
this_list = [32, 50, 655, 842, 23,102,907]  
  
this_list.sort()  
  
print(this_list)
```

Output:

```
[23, 32, 50, 102, 655, 842, 907]
```

Input:

```
thislist = ["mango", "cherry", "pineapple", "banana","apple"]  
thislist.sort(reverse = True)  
print(thislist)
```

Output:

```
['pineapple', 'mango', 'cherry', 'banana', 'apple']
```

➤ Copy List:

Input:

```
this_list = ["apple", "banana", "cherry"]  
my_list = this_list.copy()  
print(my_list)
```

Output:

```
['apple', 'banana', 'cherry']
```

➤ Join Two Lists

Input:

```
list1 = ["a", "b", "c"]  
list2 = [1, 2, 3]  
  
list3 = list1 + list2  
print(list3)
```

Output:

```
['a', 'b', 'c', 1, 2, 3]
```

Input:

```
list1 = ["a", "b" , "c"]  
list2 = [1, 2, 3]  
  
for x in list2:  
    list1.append(x)  
  
print(list1)
```

Output:

```
['a', 'b', 'c', 1, 2, 3]
```

Input:

```
list1 = ["a", "b" , "c"]  
list2 = [1, 2, 3]  
  
list1.extend(list2)  
print(list1)
```

Output:

```
['a', 'b', 'c', 1, 2, 3]
```

❑ **Tuple:**

A Tuple is a collection which is ordered and unchangeable. In python tuples are written in round brackets.

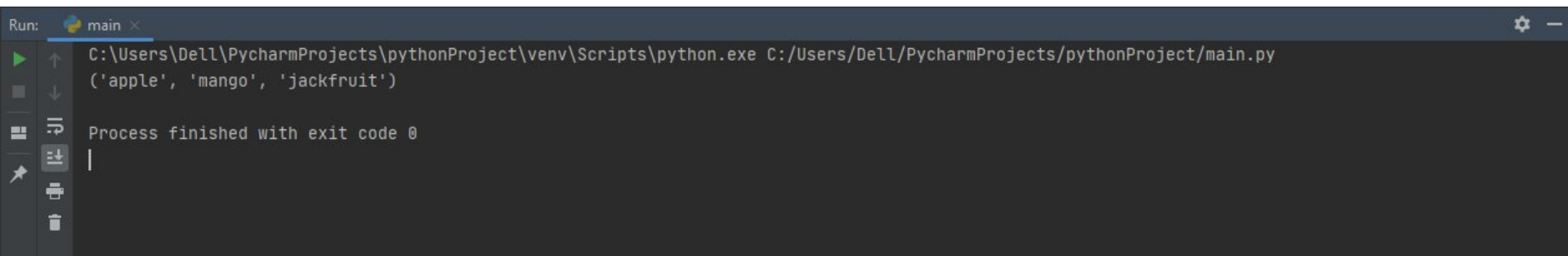
Tuples are:

1. Ordered
2. Unchangeable
3. Allow Duplicates

Input :

```
tuple=("apple","mango","jackfruit")  
print(tuple)
```

Output:

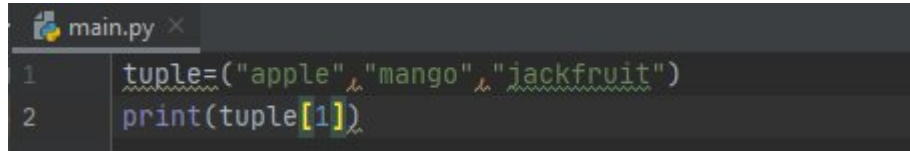


The screenshot shows a PyCharm Run console window. The title bar indicates the file being run is 'main'. The console output shows the command being executed: 'C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py'. The output of the script is '('apple', 'mango', 'jackfruit')'. Below this, it states 'Process finished with exit code 0'. The left sidebar contains standard IDE icons for running and debugging.

```
Run: main ×  
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
( 'apple', 'mango', 'jackfruit')  
Process finished with exit code 0
```

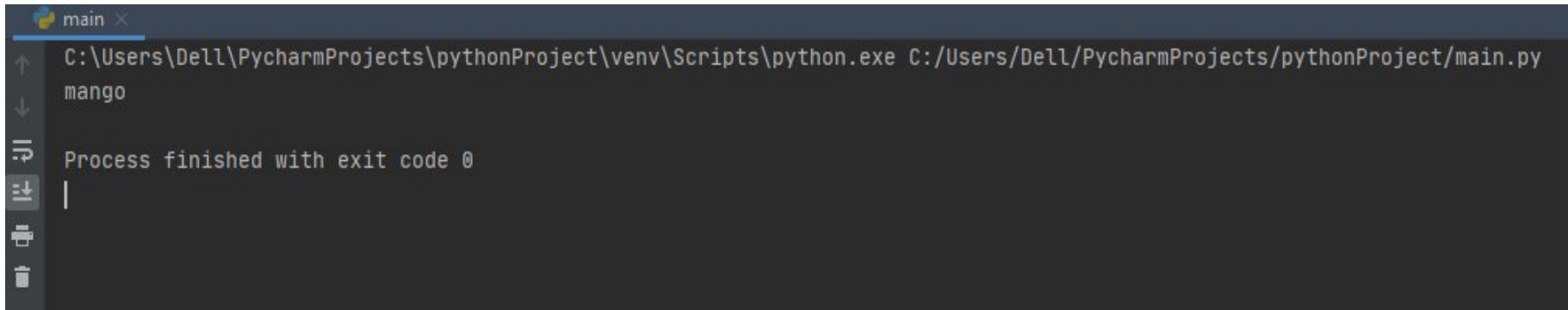
➤ Access Tuple Items:

Input:

A screenshot of a code editor window titled 'main.py'. It contains two lines of Python code: line 1 is 'tuple=("apple", "mango", "jackfruit")' and line 2 is 'print(tuple[1])'. The code is written in a dark-themed editor with syntax highlighting.

```
1 tuple=("apple", "mango", "jackfruit")
2 print(tuple[1])
```

Output:

A screenshot of a terminal window titled 'main'. It shows the command to run the script: 'C:\Users\DeLL\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/DeLL/PycharmProjects/pythonProject/main.py'. The output of the script is 'mango'. Below the output, it says 'Process finished with exit code 0'. The terminal has a dark background and a vertical toolbar on the left with icons for navigation and execution.

```
main
C:\Users\DeLL\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/DeLL/PycharmProjects/pythonProject/main.py
mango
Process finished with exit code 0
```

✓ Negative Indexing:

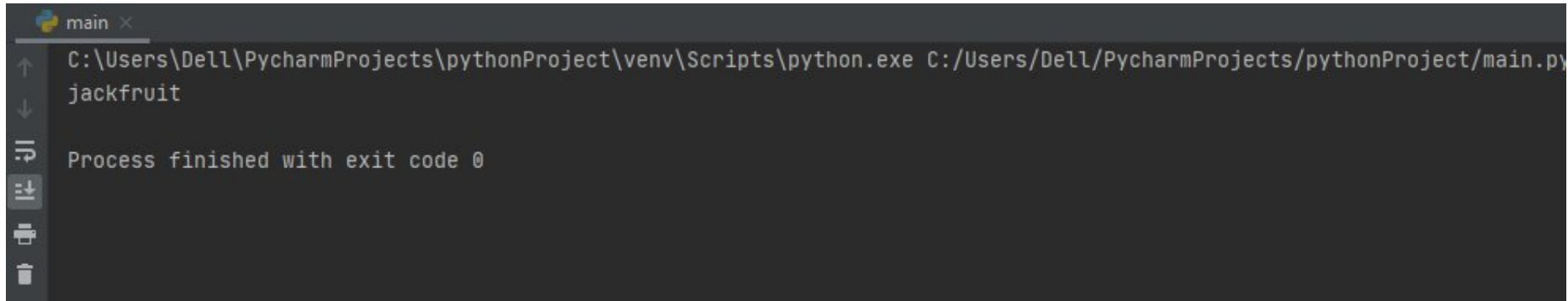
Negative indexing means beginning from the end, `-1` refers to the last item, `-2` refers to the second last item etc.

Input:

A screenshot of a code editor window titled 'main.py'. It contains two lines of Python code: line 1 is 'tuple=("apple","mango","jackfruit")' and line 2 is 'print(tuple[-1])'.

```
1 tuple=("apple","mango","jackfruit")
2 print(tuple[-1])
```

Output:

A screenshot of a terminal window titled 'main'. It shows the command to run the script and the output. The command is 'C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py'. The output is 'jackfruit'. Below the output, it says 'Process finished with exit code 0'.

```
main ×
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
jackfruit
Process finished with exit code 0
```

✓ Range of Index:

Input:

```
main.py x
1 tuple=("red" "blue" "yellow" "green" "orange" "black" "pink")
2 print(tuple[2:5])
```

Output:

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
('yellow', 'green', 'orange')

Process finished with exit code 0
```

✓ Range of Negative Index:

Input:

```
main.py x
1 tuple=("red","blue","yellow","green","orange","black","pink")
2 print(tuple[-4:-2])
```

Output:

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
('green', 'orange')

Process finished with exit code 0
```

➤ Change Tuple Value:

Input:

```
main.py x
1 x=("red","yellow","green")
2 y=list(x)
3 y[1]="blue"
4 x=tuple(y)
5 |
6 print(x)
```

Output:

```
main x
C:\Users\DeLL\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/DeLL/PycharmProjects/pythonProject/main.py
('red', 'blue', 'green')

Process finished with exit code 0
```

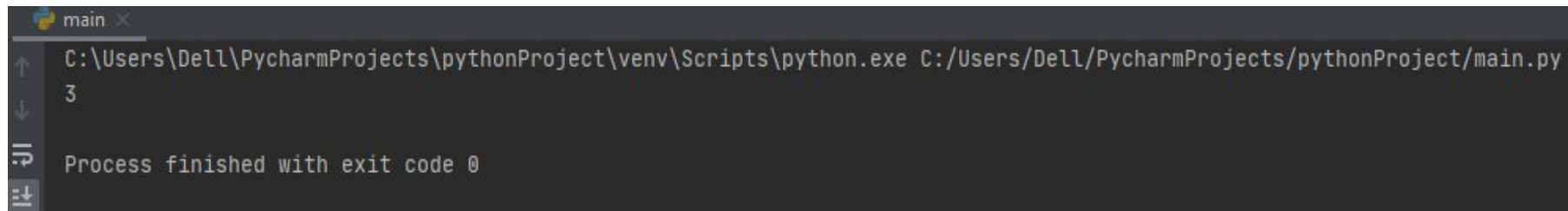
➤ Tuple Length:

To determine how many items a tuple has, use the `len()` method:

Input:

```
1 tuple = ("apple", "mango", "jackfruit")
2 print(len(tuple))
```

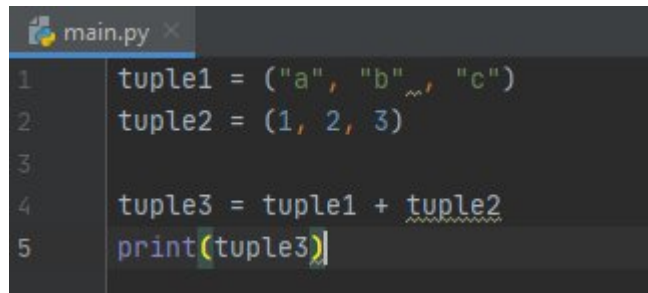
Output:



```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
3
Process finished with exit code 0
```

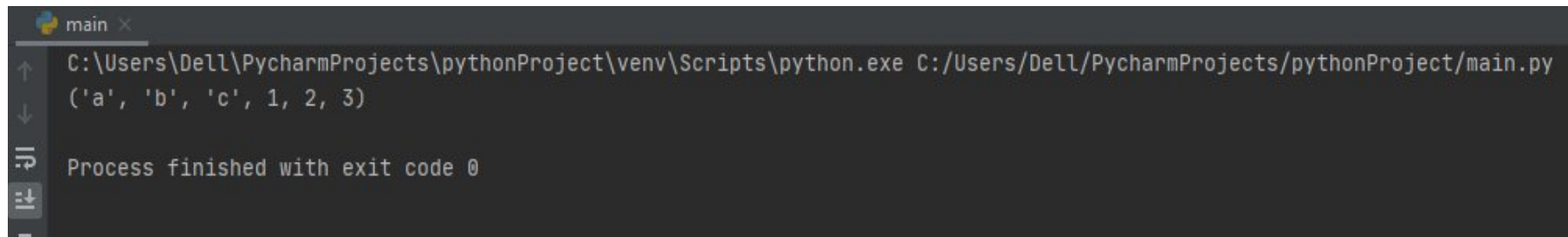
➤ Add Two Tuple:

Input:

A screenshot of a code editor window titled 'main.py'. The code defines two tuples, 'tuple1' and 'tuple2', and then concatenates them into 'tuple3' before printing it. Line numbers 1 through 5 are visible on the left side of the editor.

```
1 tuple1 = ("a", "b", "c")
2 tuple2 = (1, 2, 3)
3
4 tuple3 = tuple1 + tuple2
5 print(tuple3)
```

Output:

A screenshot of a terminal window showing the execution of the Python script. The command prompt shows the full path to the Python interpreter and the script file. The output displays the concatenated tuple. The terminal also shows the process finished with exit code 0.

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
('a', 'b', 'c', 1, 2, 3)
Process finished with exit code 0
```

➤ Multiply Tuples:

Input:

```
fruits = ("apple", "banana", "cherry")  
mytuple = fruits * 2  
  
print(mytuple)
```

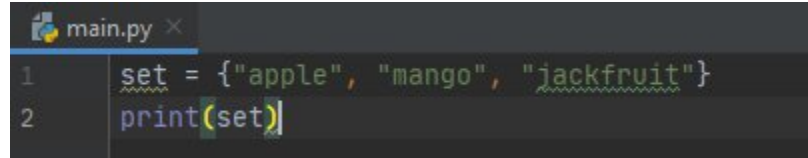
Output:

```
('apple', 'banana', 'cherry', 'apple', 'banana', 'cherry')
```

❑ Set:

A set is a collection which is unordered and unindexed. In Python, sets are written with curly brackets

Input:

A screenshot of a code editor window titled 'main.py'. It contains two lines of Python code: line 1 is 'set = {"apple", "mango", "jackfruit"}' and line 2 is 'print(set)'.

```
1 set = {"apple", "mango", "jackfruit"}
2 print(set)
```

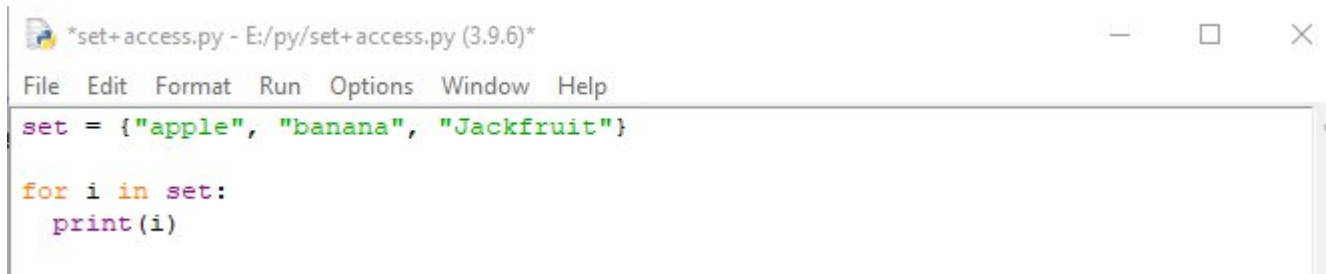
Output:

A screenshot of a terminal window titled 'main'. It shows the command prompt path 'C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/', followed by the output of the script: {'apple', 'mango', 'jackfruit'}. Below this, it says 'Process finished with exit code 0'.

```
main ×
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/
{'apple', 'mango', 'jackfruit'}
Process finished with exit code 0
```

➤ Access Set Items:

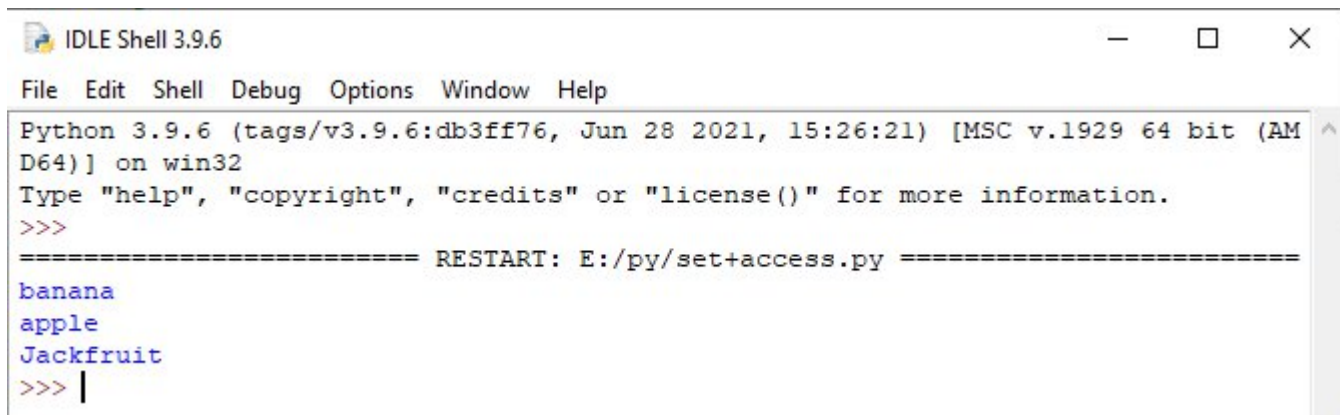
Input:



```
*set+access.py - E:/py/set+access.py (3.9.6)*
File Edit Format Run Options Window Help
set = {"apple", "banana", "Jackfruit"}

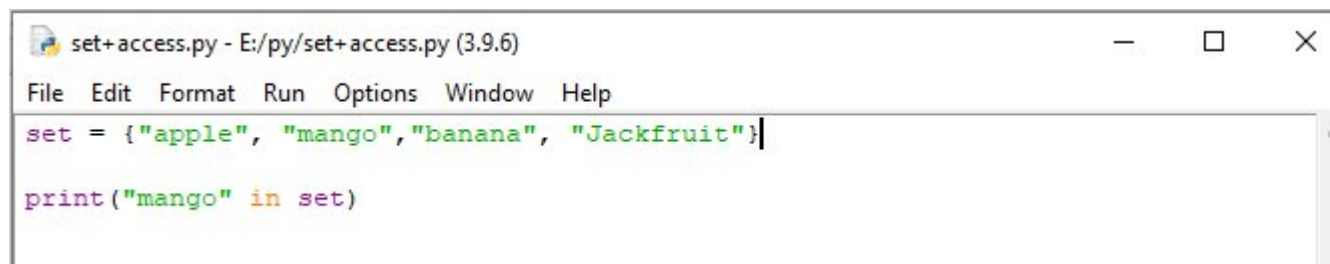
for i in set:
    print(i)
```

Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set+access.py =====
banana
apple
Jackfruit
>>> |
```

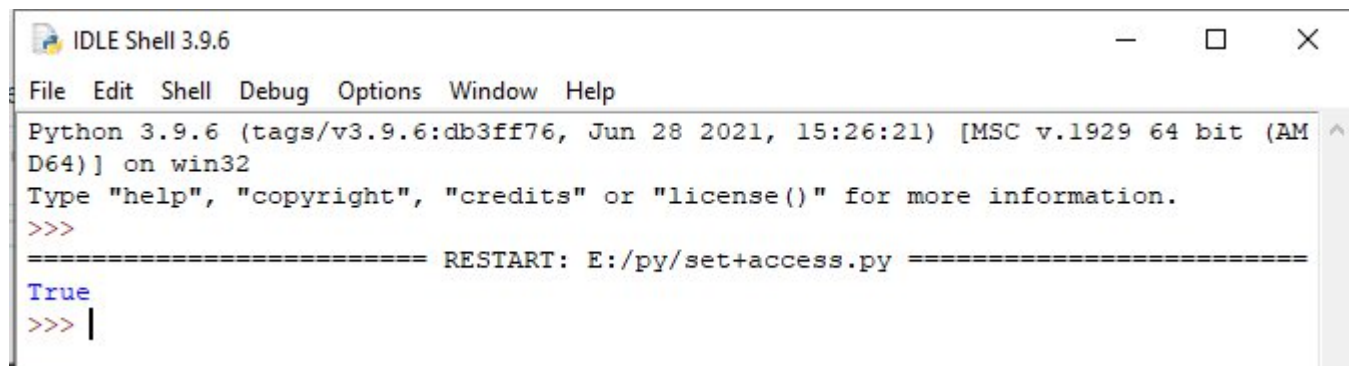
Input:



The screenshot shows a window titled "set+access.py - E:/py/set+access.py (3.9.6)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
set = {"apple", "mango", "banana", "Jackfruit"}  
  
print("mango" in set)
```

Output:



The screenshot shows the "IDLE Shell 3.9.6" window. The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The shell displays the following text:

```
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>  
===== RESTART: E:/py/set+access.py =====  
True  
>>> |
```

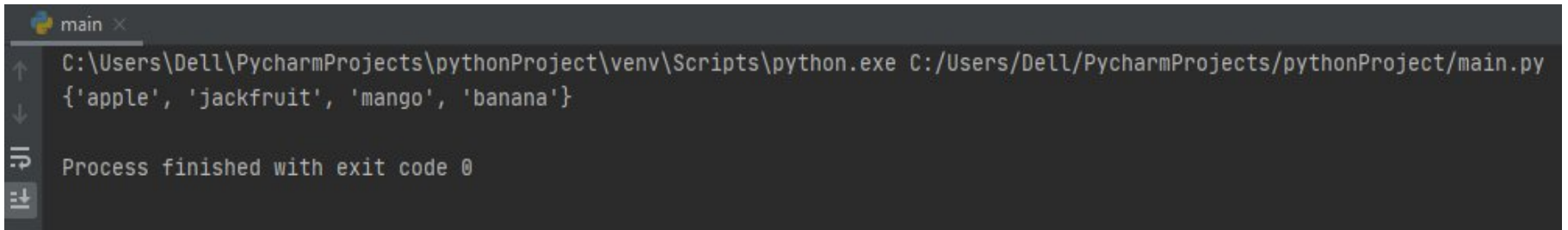
➤ Add Items:

Input:

A screenshot of a code editor window titled 'main.py'. The code consists of four lines: line 1 creates a set with 'apple', 'mango', and 'jackfruit'; line 2 adds 'banana' to the set; line 3 has a lightbulb icon; line 4 prints the set.

```
1 set = {"apple", "mango", "jackfruit"}  
2 set.add("banana")  
3   
4 print(set)
```

Output:

A screenshot of a terminal window titled 'main'. It shows the command to run the script and the resulting output, which is a set containing 'apple', 'jackfruit', 'mango', and 'banana'. The terminal also shows the process finished with exit code 0.

```
main ×  
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
{'apple', 'jackfruit', 'mango', 'banana'}  
  
Process finished with exit code 0
```

➤ Add Multiple Items:

Input:

```
main.py x
1  set = {"apple", "mango", "jackfruit"}
2  set.update(["banana", "pineapple", "watermelon", "orange"])
3
4  print(set)
```

Output:

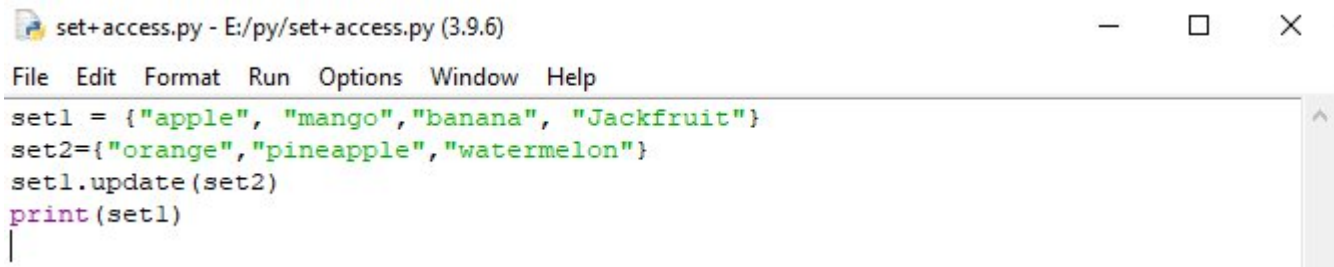
```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.p
{'mango', 'orange', 'apple', 'banana', 'jackfruit', 'watermelon', 'pineapple'}

Process finished with exit code 0
```

➤ Add Sets:

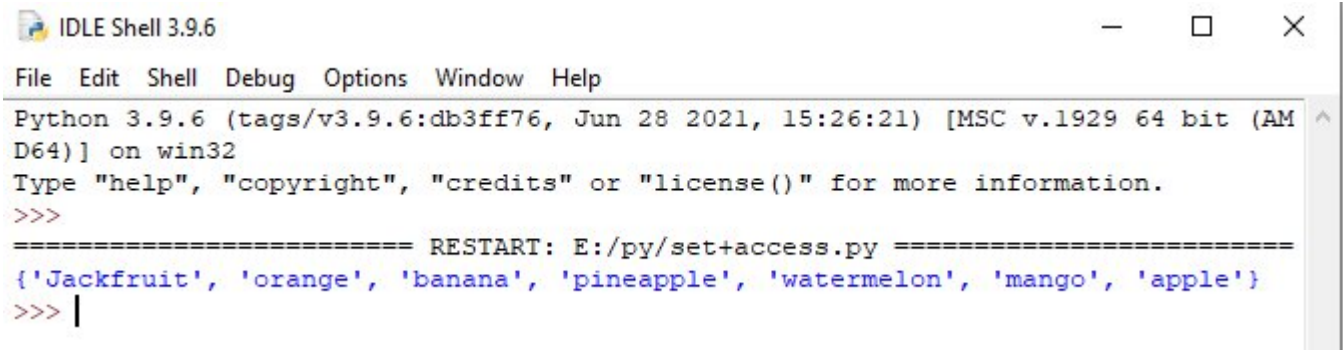
The **update ()** method inserts the items in set2 to set1.

Input:



```
File Edit Format Run Options Window Help
set1 = {"apple", "mango", "banana", "Jackfruit"}
set2={"orange", "pineapple", "watermelon"}
set1.update(set2)
print(set1)
|
```

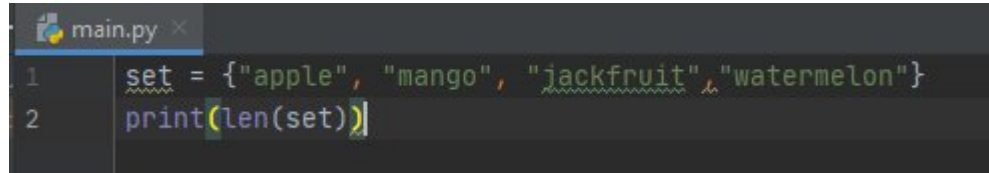
Output:



```
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set+access.py =====
{'Jackfruit', 'orange', 'banana', 'pineapple', 'watermelon', 'mango', 'apple'}
>>> |
```

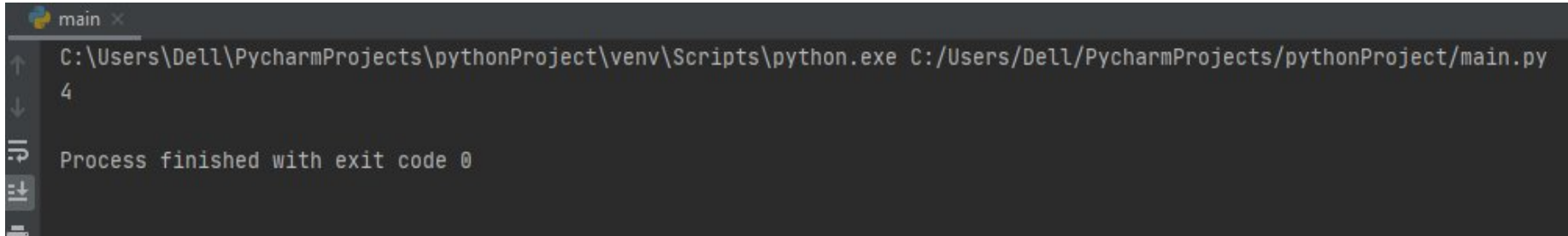
➤ Length Of a Set:

Input:

A screenshot of a code editor window titled 'main.py'. It contains two lines of Python code: a set definition and a print statement.

```
1 set = {"apple", "mango", "jackfruit", "watermelon"}  
2 print(len(set))
```

Output:

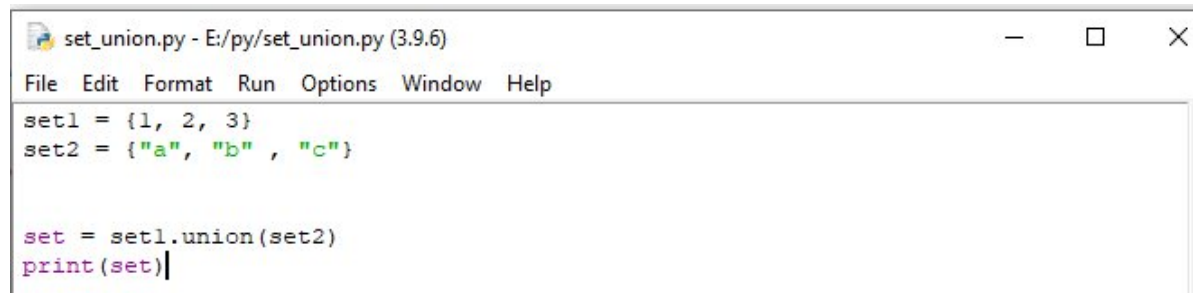
A screenshot of a terminal window titled 'main'. It shows the command to run the script and the output.

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
4  
  
Process finished with exit code 0
```

➤ Join To Sets:

The **union ()** method returns a new set with all items from both sets:

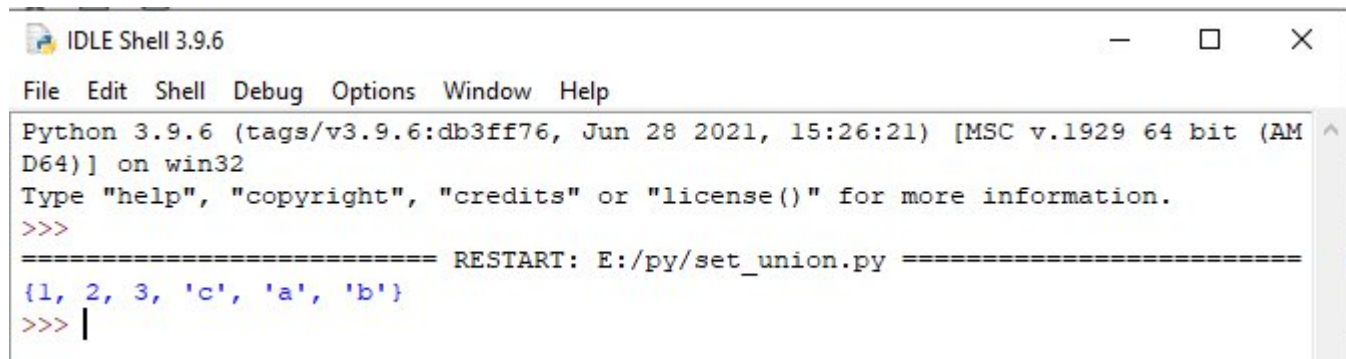
Input:

A screenshot of a text editor window titled 'set_union.py - E:/py/set_union.py (3.9.6)'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code inside is as follows:

```
set1 = {1, 2, 3}
set2 = {"a", "b", "c"}

set = set1.union(set2)
print(set)
```

Output:

A screenshot of the IDLE Shell window titled 'IDLE Shell 3.9.6'. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The output text is as follows:

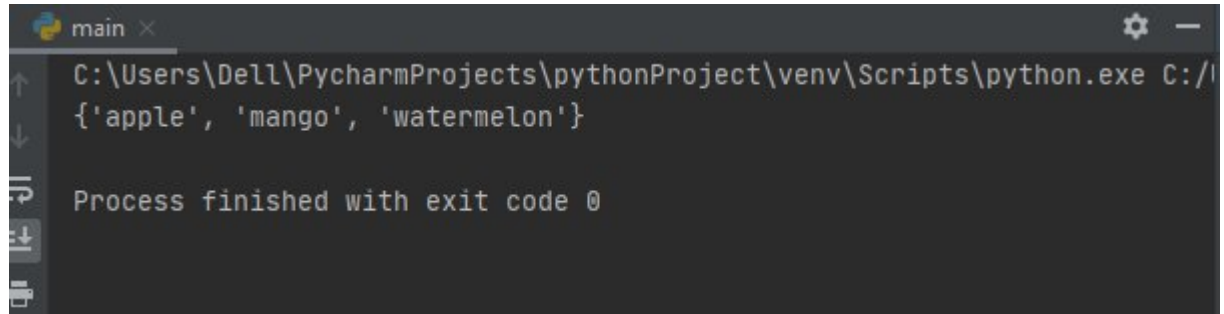
```
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:/py/set_union.py =====
{1, 2, 3, 'c', 'a', 'b'}
>>> |
```

➤ Remove an Item:

Input:

```
1  set = {"apple", "mango", "jackfruit", "watermelon"}
2  set.remove("jackfruit")
3
4  print(set)
```

Output:



The screenshot shows a terminal window titled 'main' with a dark background. The command prompt is 'C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/l'. The output is '{'apple', 'mango', 'watermelon'}'. Below the output, it says 'Process finished with exit code 0'. The terminal window has standard Windows taskbar icons (back, forward, search, etc.) on the left and a settings gear icon and a close button on the right.

```
main x
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/l
{'apple', 'mango', 'watermelon'}

Process finished with exit code 0
```

➤ Keep All, But NOT the Duplicates

Input:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
  
x.symmetric_difference_update(y)  
  
print(x)
```

Output:

```
{'google', 'banana', 'microsoft', 'cherry'}
```

➤ Keep ONLY the Duplicates

Input:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
  
x.intersection_update(y)  
  
print(x)
```

Output:

```
{'apple'}
```

Input:

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
  
z = x.intersection(y)  
  
print(z)
```

Output:

```
{'apple'}
```

❑ Python Dictionary:

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.

Example:

Input:

```
1 dict = {  
2     "name": "Hridoy",  
3     "class": "Nine",  
4     "roll": 14  
5 }  
6 print(dict)
```

Output:

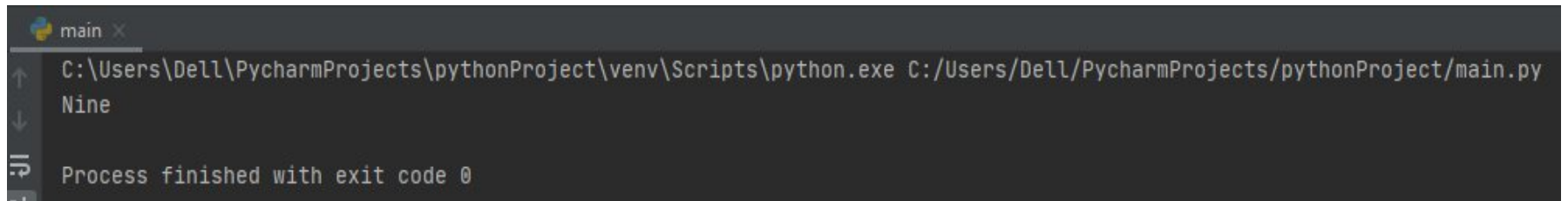
```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C  
{'name': 'Hridoy', 'class': 'Nine', 'roll': 14}  
  
Process finished with exit code 0
```

➤ Accessing Items:

Input:

```
1 dict = {  
2     "name": "Hridoy",  
3     "class": "Nine",  
4     "roll": 14  
5 }  
6 x=dict["class"]  
7 print(x)
```

Output:



```
main x  
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Nine  
Process finished with exit code 0
```

➤ Change Value:

Input:

```
home > asif > Course > PPT > python > 🐍 Untitled-1.py > ...  
1 dict={  
2     'Name': 'Hridoy',  
3     'Class': 'Nine',  
4     'Roll': 8  
5 }  
6 dict['Roll']=12  
7  
8 print(dict)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  
[Running] python -u "/home/asif/Course/PPT/python/Untitled-1.py"  
{'Name': 'Hridoy', 'Class': 'Nine', 'Roll': 12}  
  
[Done] exited with code=0 in 0.04 seconds
```

➤ Dictionary Length:

Input:

```
1 dict={  
2     "name": "Hridoy",  
3     "class": "Nine",  
4     "roll": 14  
5 }  
6 print(len(dict))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
3  
  
Process finished with exit code 0
```

➤ Adding items:

Input:

```
1 dict={
2     'Name': "Asif",
3     'Class':7,
4     'Roll': 5
5 }
6 dict['Section']='Science'
7 print(dict)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
[Running] python -u "/home/asif/Course/PPT/python/Untitled-1.py"
{'Name': 'Asif', 'Class': 7, 'Roll': 5, 'Section': 'Science'}

[Done] exited with code=0 in 0.052 seconds
```

➤ Removing Items:

Input:

```
1 dict={
2     "name": "Hridoy",
3     "class": "Nine",
4     "roll": 14
5 }
6 del dict["roll"]
7 print(dict)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe
{'name': 'Hridoy', 'class': 'Nine'}
```

```
Process finished with exit code 0
```

➤ Copy a Dictionary:

Input:

```
1 dict={
2     "name": "Hridoy",
3     "class": "Nine",
4     "roll": 14
5 }
6 copy_dict=dict.copy()
7 print(copy_dict)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Script
{'name': 'Hridoy', 'class': 'Nine', 'roll': 14}

Process finished with exit code 0
```

➤ Nested Dictionaries:

Input:

```
1 student_list={
2     "student_1":{
3         "name": "Hridoy",
4         "class": "Seven",
5         "roll": 14
6     },
7     "student_2":{
8         "name": "Arafat",
9         "class": "Seven",
10        "roll": 34
11    },
12    "student_3":{
13        "name": "Osman",
14        "class": "Seven",
15        "roll": 7
16    }
17 }
18 print(student_list)
```

Output:

```
main x
rs\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
ent_1': {'name': 'Hridoy', 'class': 'Seven', 'roll': 14}, 'student_2': {'name': 'Arafat', 'class': 'Seven', 'roll': 34}, 'student_3':

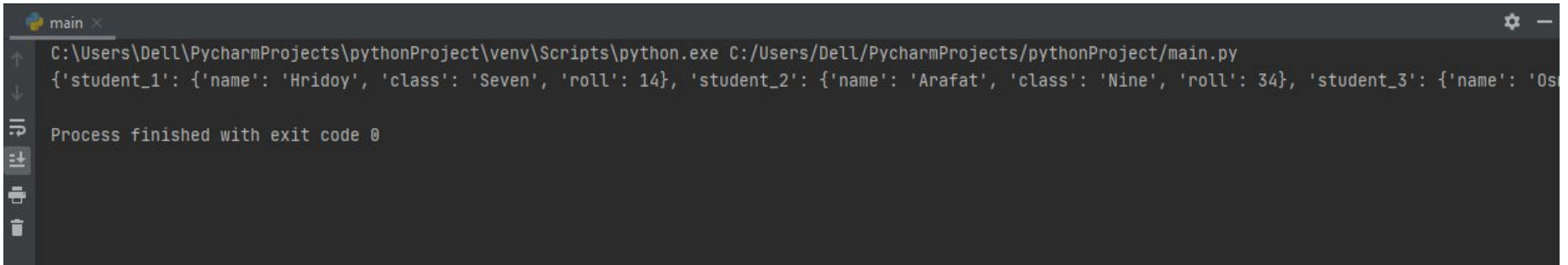
s finished with exit code 0
```

```
'student_3': {'name': 'Osman', 'class': 'Seven', 'roll': 7}}
```

Input:

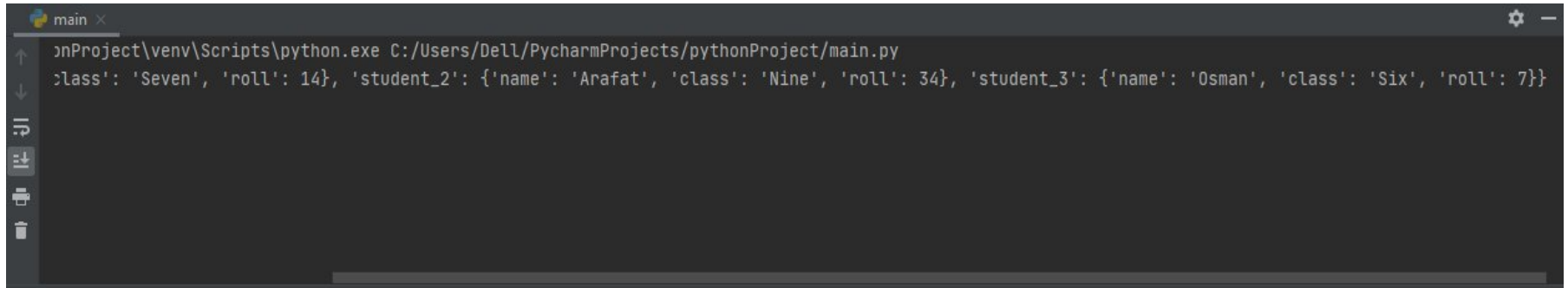
```
1 student_1= {  
2     "name": "Hridoy",  
3     "class": "Seven",  
4     "roll": 14  
5 }  
6 student_2= {  
7     "name": "Arafat",  
8     "class": "Nine",  
9     "roll": 34  
10 }  
11 student_3= {  
12     "name": "Osman",  
13     "class": "Six",  
14     "roll": 7  
15 }  
16 student_list = {  
17     "student_1": student_1,  
18     "student_2": student_2,  
19     "student_3": student_3  
20 }  
21 print(student_list)
```

Output:



The image shows a PyCharm console window with a dark theme. The title bar at the top says "main" with a close button. The console output consists of two lines: the first line is the command to run a Python script, and the second line is the output of the script. The output is a JSON object containing three student records. The console also shows the message "Process finished with exit code 0" at the bottom.

```
main x [gear icon] [minus icon]
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
{'student_1': {'name': 'Hridoy', 'class': 'Seven', 'roll': 14}, 'student_2': {'name': 'Arafat', 'class': 'Nine', 'roll': 34}, 'student_3': {'name': 'Osman', 'class': 'Six', 'roll': 7}}
Process finished with exit code 0
```



The image shows a PyCharm console window with a dark theme. The title bar at the top says "main" with a close button. The console output consists of two lines: the first line is the command to run a Python script, and the second line is the output of the script. The output is a JSON object containing three student records. The console also shows the message "Process finished with exit code 0" at the bottom.

```
main x [gear icon] [minus icon]
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
{'student_1': {'name': 'Hridoy', 'class': 'Seven', 'roll': 14}, 'student_2': {'name': 'Arafat', 'class': 'Nine', 'roll': 34}, 'student_3': {'name': 'Osman', 'class': 'Six', 'roll': 7}}
Process finished with exit code 0
```

➤ Loop Dictionaries:

Input:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
for x in thisdict:  
    print(x)
```

Output:

```
brand  
model  
year
```

Input:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
for x in thisdict:  
    print(thisdict[x])
```

Output:

```
Ford  
Mustang  
1964
```

14. Python If..Else:

Python condition and if statement:

- Equal: $a==b$
- Not Equal: $a!=b$
- Greater than: $a>b$
- Greater or equal than: $a>=b$
- Less than: $a<b$
- Less or equal than: $a<=b$

Example:

If statement:

```
1 a = 23
2 b = 9
3 if a > b:
4     print("a is greater than b")
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b
```

```
Process finished with exit code 0
```

Else:

Input:

```
1 a = 399
2 b = 206
3 if a > b:
4     print("a is greater than b")
5 else:
6     print("a is not greater than b")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b

Process finished with exit code 0
```

Elif:

Input:

```
1 a = 200
2 b = 300
3 if a > b:
4     print("a is greater than b")
5 elif a == b:
6     print("a and b are equal")
7 else:
8     print("b is greater than a")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
b is greater than a
```

```
Process finished with exit code 0
```

ShortHand If:

Input:

```
1 a=76
2 b=39
3 if a > b: print("a is greater than b")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b

Process finished with exit code 0
```

ShortHand if..Else:

Input:

```
1 a = 322
2 b = 233
3 print("a is greater than b") if a > b else print("a is not greater than b")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
a is greater than b
```

```
Process finished with exit code 0
```

And:

Input:

```
1 a = 1003
2 b = 4076
3 c = 520
4 if b > a and a > c:
5     print("Both conditions are True")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
Both conditions are True

Process finished with exit code 0
```

Or:

Input:

```
1 a = 300
2 b = 200
3 c = 500
4 if a > b or a > c:
5     print("At least one of the conditions is True")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
At least one of the conditions is True
```

```
Process finished with exit code 0
```

Nested If:

Input:

```
1 mark = 76
2 if mark < 35:
3     print("Grade is F")
4 if mark >= 35 and mark < 40:
5     print("Grade is D")
6 if mark >= 40 and mark < 50:
7     print("Grade is C")
8 if mark >= 50 and mark < 60:
9     print("Grade is B")
10 if mark >= 60 and mark < 70:
11     print("Grade is A-")
12 if mark >= 70 and mark < 80:
13     print("Grade is A")
14 if mark >= 80:
15     print("Grade is A+")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
Grade is A
```

```
Process finished with exit code 0
```

Nested if using Input Method: Input:

```
1  mark =int(input("mark?"))
2  if mark<35:
3      print("Grade is F")
4  if mark >= 35 and mark <40:
5      print("Grade is D")
6  if mark >= 40 and mark <50 :
7      print("Grade is C")
8  if mark >= 50 and mark <60:
9      print("Grade is B")
10 if mark >= 60 and mark <70:
11     print("Grade is A-")
12 if mark >= 70 and mark <80:
13     print("Grade is A")
14 if mark >= 80:
15     print("Grade is A+")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
mark?56
Grade is B

Process finished with exit code 0
```

15. Python Loop:

- While Loop
- For Loop

While Loop:

Input:

```
1 i = 1
2 while i < 5:
3     print(i)
4     i += 1
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
1
2
3
4

Process finished with exit code 0
```

Else In While Loop:

Input:

```
1 i = 1
2 while i < 5:
3     print(i)
4     i += 1
5 else:
6     print("i is no longer less than 5")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
1
2
3
4
i is no longer less than 5

Process finished with exit code 0
```

- **For Loop:**

Input:

```
1  fruits = ["apple", "mango", "jackfruit"]
2  for x in fruits:
3      print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango
jackfruit
```

```
Process finished with exit code 0
```

Looping Through a String

Input:

```
1 for x in "mango":  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
m  
a  
n  
g  
o  
  
Process finished with exit code 0
```

Else in For Loop

Input:

```
1 for x in range(6):  
2     print(x)  
3 else:  
4     print("Finished!")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
0  
1  
2  
3  
4  
5  
Finished!
```

Nested Loops

Input:

 nested.py - D:\asif-office-work\python_code\nested.py (3.10.2)

File Edit Format Run Options Window Help

```
shade = ["light", "deep"]  
colour = ["red", "blue", "green"]
```

```
for x in shade:  
    for y in colour:  
        print(x, y)
```

Output:

Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\nested.py =====

```
light red  
light blue  
light green  
deep red  
deep blue  
deep green
```

16. Python Statement:

Pass Statement:

Input:

```
1 a = 499
2 b = 200
3
4 if b > a:
5     pass
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
Process finished with exit code 0
```

- **pass Statement in For loop**

`for` loops cannot be empty, but if you for some reason have a `for` loop with no content, put in the `pass` statement to avoid getting an error.

```
1 for x in [0, 1, 2, 3, 4]:  
2     pass
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
Process finished with exit code 0
```

- **The continue Statement**

With the `continue` statement we can stop the current iteration of the loop, and continue with the next:

Example:

Do not print **jackfruit** :

Input:

```
1  fruits = ["apple", "mango", "jackfruit", "orange"]
2  for x in fruits:
3      if x == "jackfruit":
4          continue
5      print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango
orange

Process finished with exit code 0
```

Continue Statement:

Input:

```
1 i = 0
2 while i < 5:
3     i += 1
4     if i == 3:
5         continue
6     print(i)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
```

```
1
2
4
5
```

```
Process finished with exit code 0
```

- **Break Statement:**

Input:

```
1   i = 1
2   while i < 5:
3       print(i)
4       if i == 3:
5           break
6       i += 1
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
1
2
3

Process finished with exit code 0
```

break Statement in For Loop:

Input:

```
1  fruits = ["apple", "mango", "jackfruit", "orange"]
2  for x in fruits:
3      print(x)
4      if x == "jackfruit":
5          break
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango
jackfruit

Process finished with exit code 0
```

Exit the loop when `x` is "banana", but this time the break comes before the print:

Input:

```
1  fruits = ["apple", "mango", "jackfruit", "orange"]
2  for x in fruits:
3      if x == "jackfruit":
4          break
5      print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
apple
mango

Process finished with exit code 0
```

17. The range() Function

Input:

```
1 for x in range(5):  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
0  
1  
2  
3  
4  
  
Process finished with exit code 0
```

Example:

Input:

```
1 for x in range(2, 7):  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
2  
3  
4  
5  
6  
  
Process finished with exit code 0
```

Example

Input:

```
1 for x in range(5, 50, 15):  
2     print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
5  
20  
35  
  
Process finished with exit code 0
```

18.Function:

Creating a function:

```
1 def my_function():  
2     print("Hello world!")
```

Calling a function:

```
1 def my_function():  
2     print("Hello world!")  
3  
4     my_function()
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Hello world!
```

```
Process finished with exit code 0
```

❑ Arguments

Information can be passed into functions as arguments.

Input:

```
def my_function(fname):  
    print("Hello "+fname)  
  
my_function("asif")
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
Hello asif  
  
Process finished with exit code 0
```

❑ Parameters or Arguments?

The terms *parameter* and *argument* can be used for the same thing: **information that are passed into a function.**

From a function's perspective:

A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

Function definition

`def fname(arg1,arg2):`
`statement(s)`

Formal parameters

Function call

`fname(a,b)`

Actual parameters(arguments)

5 Types of Arguments in Python Function Definition:

1. Default Arguments
2. Keyword Arguments
3. Positional Arguments
4. Arbitrary Arguments
5. Arbitrary Keyword Arguments

1. Default arguments:

- Default arguments are values that are provided while defining functions.
- The assignment operator = is used to assign a default value to the argument.
- Default arguments become optional during the function calls.
- If we provide a value to the default arguments during function calls, it overrides the default value.
- The function can have any number of default arguments
- Default arguments should follow non-default arguments.

```
def add(a,b=5,c=10):  
    return (a+b+c)
```

```
print(add(3))  
#Output:18
```

```
print(add(3,4))  
#Output:17
```

❑ Default Parameter Value

```
def my_function(country = "Bangladesh"):  
    print("I am from " + country)
```

```
my_function("Sweden")  
my_function("India")  
my_function()  
my_function("America")
```

```
I am from Sweden  
I am from India  
I am from Bangladesh  
I am from America
```

2. Keyword Arguments:

Functions can also be called using keyword arguments of the form `kwarg=value`.

During a function call, values passed through arguments need not be in the order of parameters in the function definition. This can be achieved by keyword arguments. But all the keyword arguments should match the parameters in the function definition.

```
def add(a,b=5,c=10):  
    return (a+b+c)
```

```
print (add(b=10,c=15,a=20))  
#Output:45
```

```
print (add(a=10))  
#Output:25
```

3. Positional Arguments

During a function call, values passed through arguments should be in the order of parameters in the function definition. This is called **positional arguments**.

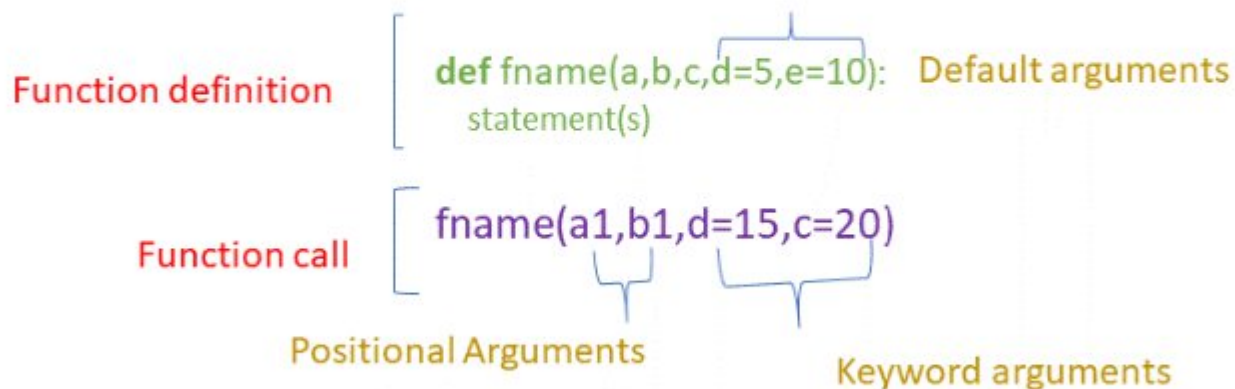
Keyword arguments should follow positional arguments only.

```
def add(a,b,c):  
    return (a+b+c)
```

```
print (add(10,20,30))  
#Output:60
```

```
print (add(10,c=30,b=20))  
#Output:60
```

default vs positional vs keyword arguments:



Different Types of Arguments:

```
# default arguments
print("Hello {}, your balance is {}".format("Hridoy", 10035))

# positional arguments
print("Hello {0}, your balance is {1}".format("Hridoy", 430.2346))

# keyword arguments
print("Hello {name}, your balance is {blc}".format(name="Hridoy", blc=230.23))

# mixed arguments
print("Hello {0}, your balance is {blc}".format("Hridoy", blc=2500))
```

```
==== RESTART: D:\Asif-office-work\Batch\Python With !
Hello Hridoy, your balance is 10035.
Hello Hridoy, your balance is 430.2346.
Hello Hridoy, your balance is 230.23.
Hello Hridoy, your balance is 2500.
```

❑ Variable-length arguments

Variable-length arguments are also known as **arbitrary arguments**. If we don't know the number of arguments needed for the function in advance, we can use arbitrary arguments

Two types of arbitrary arguments

- 1.arbitrary positional arguments
- 2.arbitrary keyword arguments

4. arbitrary positional arguments:

For arbitrary positional argument, an **asterisk (*)** is placed before a parameter in function definition which can hold non-keyword variable-length arguments. These arguments will be wrapped up in a **tuple**. Before the variable number of arguments, zero or more normal arguments may occur.

```
def add(*b):  
    result=0  
    for i in b:  
        result=result+i  
    return result  
  
print (add(1,2,3,4,5))  
#Output:15  
  
print (add(10,20))  
#Output:30
```

5.arbitrary keyword arguments:

For arbitrary positional argument, a **double asterisk (**)** is placed before a parameter in a function which can hold keyword variable-length arguments.

Example:

```
def fn(**a):  
    for i in a.items():  
        print (i)  
fn(numbers=5,colors="blue",fruits="apple")  
'''  
Output:  
( 'numbers', 5)  
( 'colors', 'blue')  
( 'fruits', 'apple')  
'''
```

Input:

```
home > asif > Course > PPT > python > Untitled-1.py > ...
1  def process_data(**kwargs):
2      for key, value in kwargs.items():
3          print(f'{key}: {value}')
4
5  # Calling the function with arbitrary keyword arguments
6  process_data(name='John', age=25, city='New York')
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
[Running] python -u "/home/asif/Course/PPT/python/Untitled-1.py"
name: John
age: 25
city: New York

[Done] exited with code=0 in 0.03 seconds
```

Passing a List as an Argument

Input:

```
def my_function(fruits):  
    for x in fruits:  
        print(x)  
  
fruits = ["apple", "banana", "cherry"]  
  
my_function(fruits)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  
  
Try the new cross-platform PowerShell https://aka.ms/pscore6  
  
PS C:\Users\Microland IT> & "C:/Users/Microland IT/AppData/Local/  
apple  
banana  
cherry
```

Return Values

To let a function return a value, use the `return` statement:

Input:

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))  
print(my_function(12))
```

Output:

```
C:\Users\DeLL\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/DeLL/PycharmProjects/pythonProject/main.py  
15  
25  
60  
  
Process finished with exit code 0
```

19. Python Lambda:

Input:

```
add=lambda x: x+5  
print(add(3))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
8  
  
Process finished with exit code 0
```

Input:

```
a = lambda x, y : x ** y  
print(a(5, 3))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
125  
|  
Process finished with exit code 0
```

Input:

```
def my_function(n):  
    return lambda x : x * n  
  
my_doubler = my_function(2)  
  
print(my_doubler(10))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
20  
  
Process finished with exit code 0
```

Input:

```
def my_function(n):  
    return lambda x : x * n  
  
my_doubler = my_function(2)  
my_tripler = my_function(3)  
  
print(my_doubler(12))  
print(my_tripler(10))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
24  
30  
  
Process finished with exit code 0
```

20.Array:

Input:

```
fruits = ["apple", "mango", "jackfruit"]
```

```
fruits = ["apple", "mango", "jackfruit"]  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'jackfruit']  
  
Process finished with exit code 0
```

Input:

```
fruit1="apple"  
fruit2="mango"  
fruit3="jackfruit"  
  
fruits=[fruit1,fruit2,fruit3]  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'jackfruit']  
  
Process finished with exit code 0
```

Access the Elements of an Array

Input:

```
fruits=["apple","mango","jackfruit"]  
print(fruits[2])
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
jackfruit
```

```
Process finished with exit code 0
```

Modify the value of array item:

Input:

```
fruits=["apple","mango","jackfruit"]  
fruits[2]="banana"  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'banana']  
  
Process finished with exit code 0
```

Length of an array:

Input:

```
fruits=["apple","mango","jackfruit"]  
print(len(fruits))
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
3
```

```
Process finished with exit code 0
```

Looping Array Elements

Input:

```
fruits=["apple","mango","jackfruit"]  
for x in fruits:  
    print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
apple  
mango  
jackfruit  
  
Process finished with exit code 0
```

Adding Array Elements

You can use the `append()` method to add an element to an array.

Input:

```
fruits=["apple","mango","jackfruit"]  
fruits.append("orange")  
  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'jackfruit', 'orange']  
  
Process finished with exit code 0
```

Removing Array Elements

You can use the `pop()` method to remove an element from the array.

Input:

```
fruits=["apple","mango","jackfruit","watermelon"]  
fruits.pop(2)  
  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'mango', 'watermelon']  
  
Process finished with exit code 0
```

You can also use the `remove()` method to remove an element from the array.

Input:

```
fruits=["apple","mango","jackfruit","watermelon"]  
fruits.remove("mango")  
  
print(fruits)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
['apple', 'jackfruit', 'watermelon']  
  
Process finished with exit code 0
```

21. Python math:

Python has a set of built-in math functions, including an extensive math module, that allows you to perform mathematical tasks on numbers.

Built-in Math Functions:

The `min()` and `max()` functions can be used to find the lowest or highest value in an iterable:

Input:

```
x = max(5, 10, 25)
y = min(5, 10, 25)

print("max number is ",x)
print("min number is ",y)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
max number is  25
min number is  5

Process finished with exit code 0
```

The `abs()` function returns the absolute (positive) value of the specified number:

Input:

```
x = abs(-13.62)
print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
13.62

Process finished with exit code 0
```

The `pow(x, y)` function returns the value of x to the power of y (x^y).

Input:

```
x = pow(5, 3)
print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
125

Process finished with exit code 0
```

The Math Module

Python has also a built-in module called `math`, which extends the list of mathematical functions.

To use it, you must import the `math` module:

```
import math
```

When you have imported the `math` module, you can start using methods and constants of the module.

The `math.sqrt()` method for example, returns the square root of a number:

```
import math
x = math.sqrt(49)
print(x)
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
7.0
```

```
Process finished with exit code 0
```

The `math.ceil()` method rounds a number upwards to its nearest integer, and the `math.floor()` method rounds a number downwards to its nearest integer, and returns the result:

```
import math

x = math.ceil(2.6)
y = math.floor(2.6)

print(x) # returns 3
print(y) # returns 2
```

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py
3
2

Process finished with exit code 0
```

The `math.pi` constant, returns the value of PI (3.14...):

Input:

```
import math  
x = math.pi  
print(x)
```

Output:

```
C:\Users\Dell\PycharmProjects\pythonProject\venv\Scripts\python.exe C:/Users/Dell/PycharmProjects/pythonProject/main.py  
3.141592653589793
```

```
Process finished with exit code 0
```

22.Python Classes/Objects

Python is an object oriented programming language.

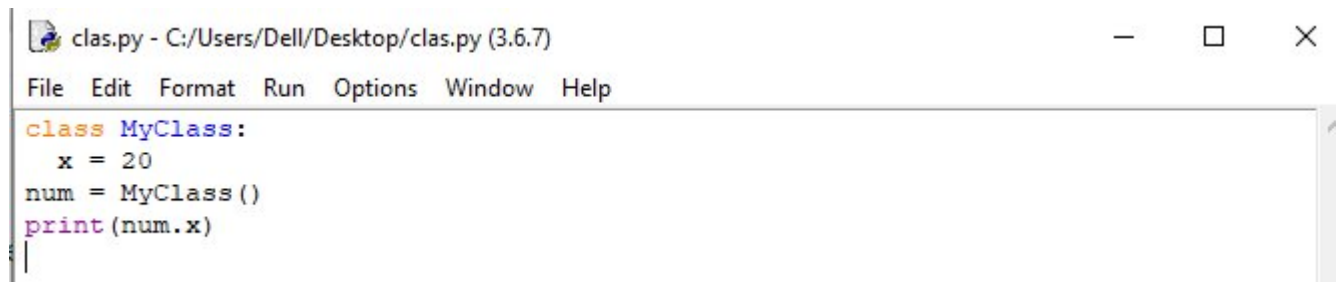
Almost everything in Python is an object, with its properties and methods.

Create a Class

To create a class, use the keyword **class**:

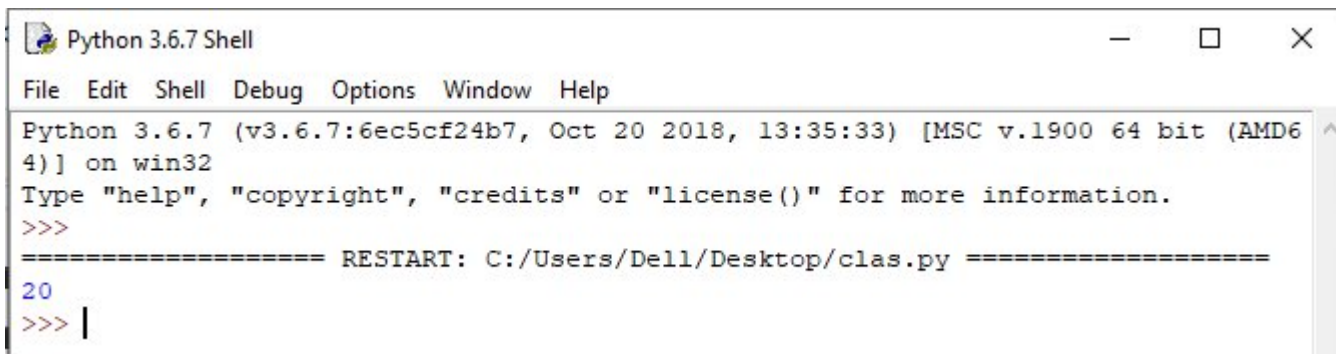
Example

Input:



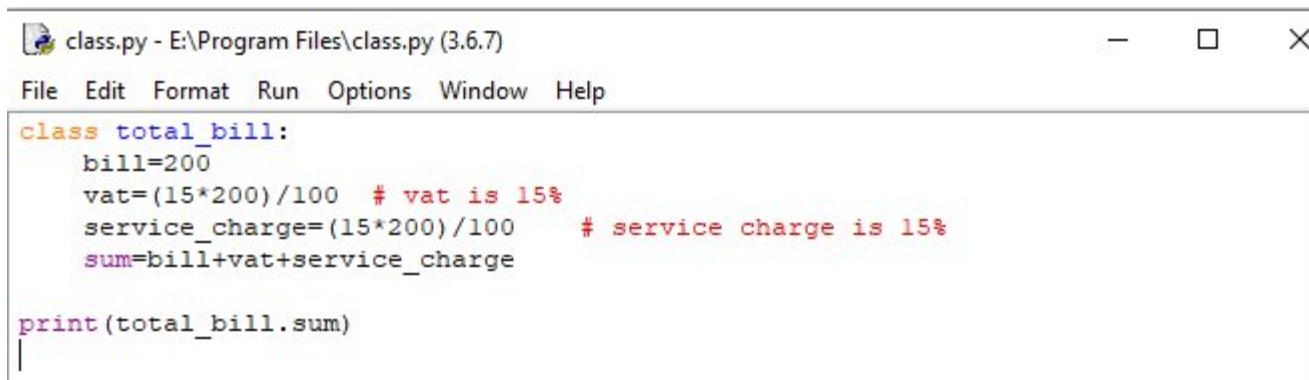
```
class MyClass:
    x = 20
num = MyClass()
print(num.x)
```

Output:



```
Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/clas.py =====
20
>>> |
```

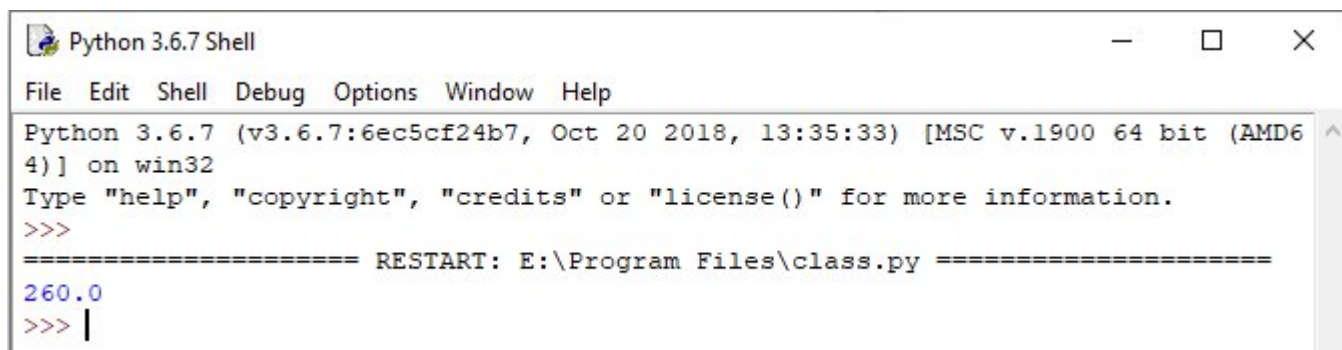
Input:



```
class total_bill:
    bill=200
    vat=(15*200)/100 # vat is 15%
    service_charge=(15*200)/100 # service charge is 15%
    sum=bill+vat+service_charge

print(total_bill.sum)
```

Output:



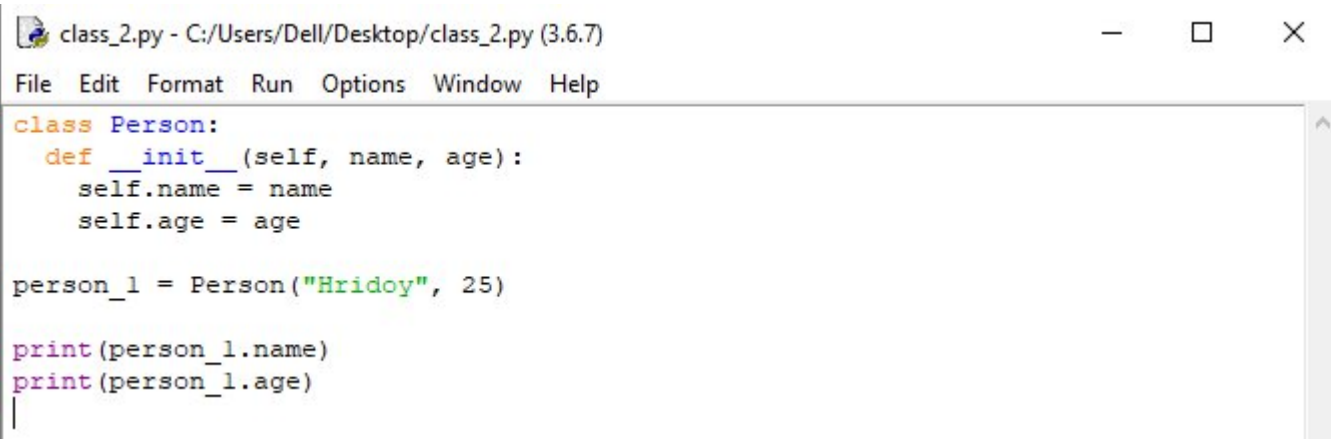
```
Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: E:\Program Files\class.py =====
260.0
>>> |
```

The `__init__()` Function

Input:

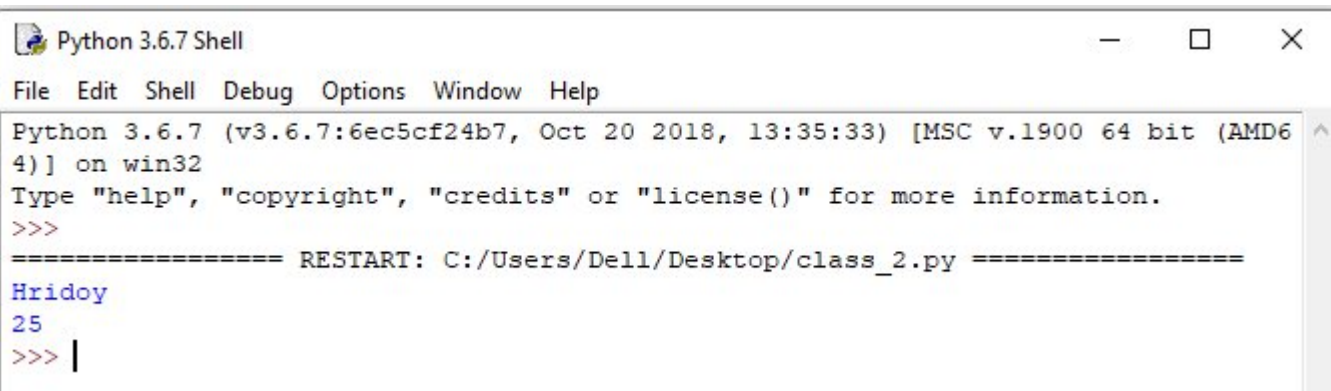


```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

person_1 = Person("Hridoy", 25)

print(person_1.name)
print(person_1.age)
```

Output:



```
Python 3.6.7 Shell

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Hridoy
25
>>> |
```

Input:

```
class_2.py - C:/Users/Dell/Desktop/class_2.py (3.6.7)
File Edit Format Run Options Window Help

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
person_1.myfunc()
```

Output:

```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Hello my name is Hridoy
>>> |
```

Input:

```
class_2.py - C:/Users/Dell/Desktop/class_2.py (3.6.7)
File Edit Format Run Options Window Help

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
person_1.name='Mahmud' # Modify Object Properties
person_1.myfunc()
```

Output:

```
Python 3.6.7 Shell
File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Hello my name is Mahmud
>>> |
```

Input:

```
class_2.py - C:/Users/Dell/Desktop/class_2.py (3.6.7)
File Edit Format Run Options Window Help

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

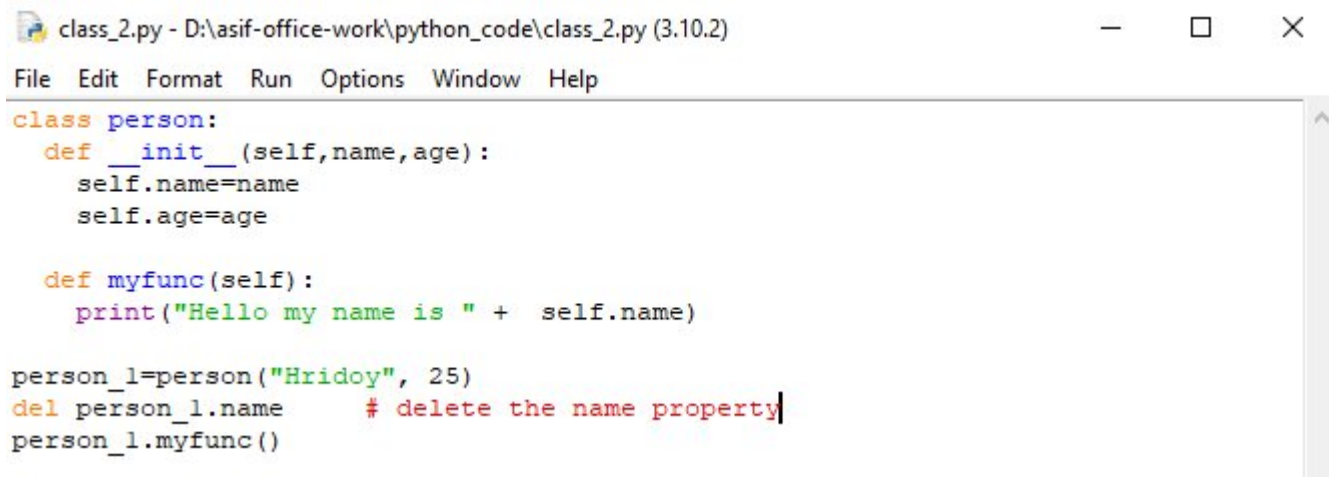
person_1 = Person("Hridoy", 25)
del person_1.age          #Delete the age property |
person_1.myfunc()
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Hello my name is Hridoy
|
```

Input:



```
class_2.py - D:\asif-office-work\python_code\class_2.py (3.10.2)
File Edit Format Run Options Window Help

class person:
    def __init__(self,name,age):
        self.name=name
        self.age=age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1=person("Hridoy", 25)
del person_1.name      # delete the name property
person_1.myfunc()
```

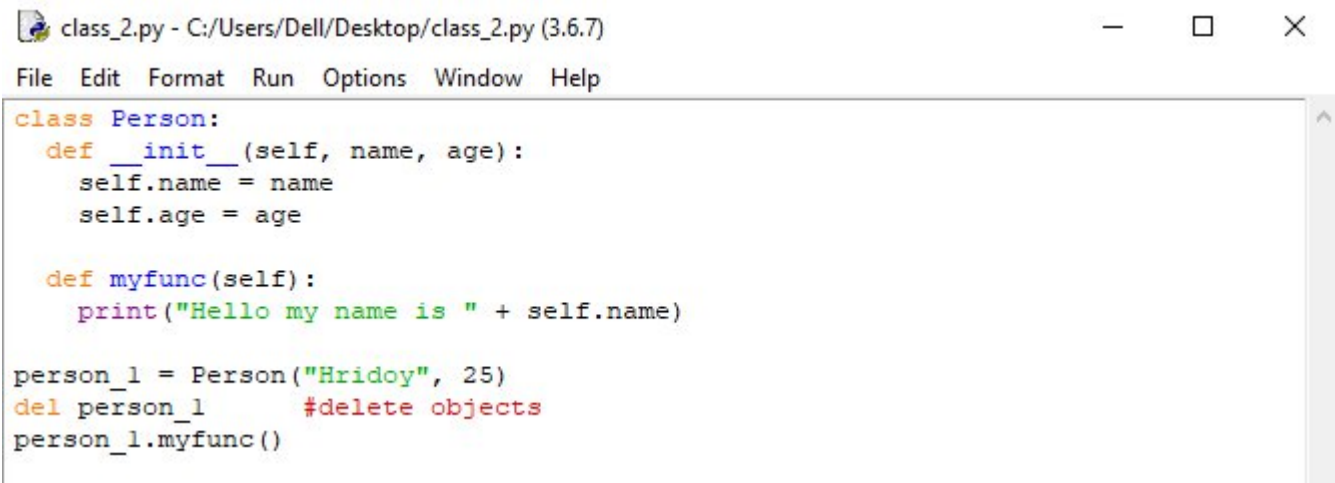
Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Hello my name is Hridoy

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Traceback (most recent call last):
  File "D:\asif-office-work\python_code\class_2.py", line 11, in <module>
    person_1.myfunc()
  File "D:\asif-office-work\python_code\class_2.py", line 7, in myfunc
    print("Hello my name is " + self.name)
AttributeError: 'person' object has no attribute 'name'
```

Input:

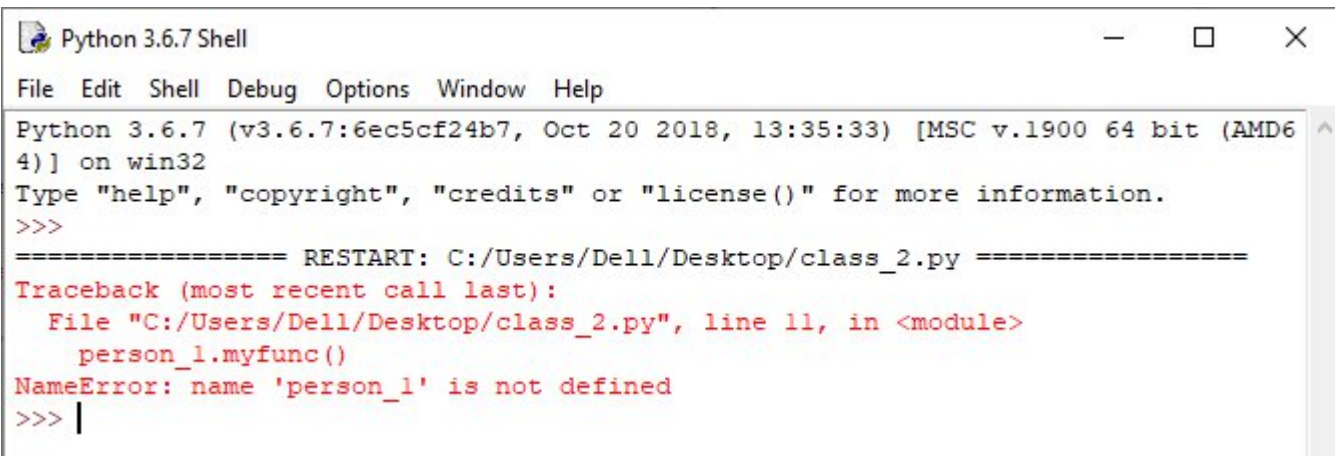


```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

person_1 = Person("Hridoy", 25)
del person_1      #delete objects
person_1.myfunc()
```

Output:



```
Python 3.6.7 Shell

File Edit Shell Debug Options Window Help

Python 3.6.7 (v3.6.7:6ec5cf24b7, Oct 20 2018, 13:35:33) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/Dell/Desktop/class_2.py =====
Traceback (most recent call last):
  File "C:/Users/Dell/Desktop/class_2.py", line 11, in <module>
    person_1.myfunc()
NameError: name 'person_1' is not defined
>>> |
```

23.Python Inheritance:

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Create a Parent Class

Any class can be a parent class, so the syntax is the same as creating any other class:

Input:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

x = Person("Salman", "Mahmud")
x.printname()
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Salman Mahmud
```

Create a Child Class

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def print_name(self):
        print(self.firstname, self.lastname)

class child_class(Person):
    pass

x = child_class("Hridoy", "khan")
x.print_name()
```

Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====

Hridoy khan

|

24.Python Iterators:

An iterator is an object that contains a countable number of values.

An iterator is an object that can be iterated upon, meaning that you can traverse through all the values.

Technically, in Python, an iterator is an object which implements the iterator protocol, which consist of the methods `__iter__()` and `__next__()`.

Iterating Through an Iterator

Input:

```
my_str = "apple"
myit = iter(my_str)

print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
a
p
p
l
e
|
```

Input:

```
my_tuple = ("apple", "mango", "jackfruit")
myit = iter(my_tuple)

print(next(myit))
print(next(myit))
print(next(myit))
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
apple
mango
jackfruit
|
```



```
# define a list
my_list = [4, 7, 0]

# create an iterator from the list
iterator = iter(my_list)

# get the first element of the iterator
print(next(iterator)) # prints 4

# get the second element of the iterator
print(next(iterator)) # prints 7

# get the third element of the iterator
print(next(iterator)) # prints 0
```

Output

```
4
7
0
```

Iterate using For loop:

Input:

```
mytuple = ("apple", "jackfruit", "mango")

for x in mytuple:
    print(x)
|
```

Output:

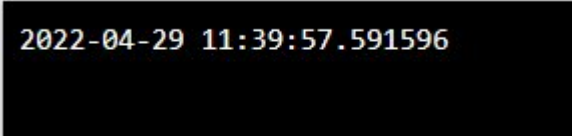
```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
apple
jackfruit
mango
|
```

25. Python Datetime

A date in Python is not a data type of its own, but we can import a module named **datetime** to work with dates as date objects.

```
import datetime  
  
x = datetime.datetime.now()  
  
print(x)
```



```
2022-04-29 11:39:57.591596
```

The date contains year, month, day, hour, minute, second, and microsecond.

```
import datetime

x = datetime.datetime.now()

print(x.year)
print(x.strftime("%A"))
```

2022
Friday

```
import datetime  
  
x = datetime.datetime(2020, 5, 17)  
  
print(x)
```

```
2020-05-17 00:00:00
```

❑ The strftime() Method

The `datetime` object has a method for formatting date objects into readable strings.

The method is called `strftime()`, and takes one parameter, `format`, to specify the format of the returned string:

```
import datetime

x = datetime.datetime(2018, 6, 1)

print(x.strftime("%B"))
```

June

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018

Directive	Description	Example
%H	Hour 00-23	17
%I	Hour 00-12	05
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	08
%f	Microsecond 000000-999999	548513
%z	UTC offset	+0100
%Z	Timezone	CST
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as the first day of week, 00-53	52

Directive	Description	Example
%W	Week number of year, Monday as the first day of week, 00-53	52
%c	Local version of date and time	Mon Dec 31 17:41:00 2018
%C	Century	20
%x	Local version of date	12/31/18
%X	Local version of time	17:41:00
%%	A % character	%
%G	ISO 8601 year	2018
%u	ISO 8601 weekday (1-7)	1
%V	ISO 8601 weeknumber (01-53)	01

26.Exception / Try Except:

The `try` block lets you test a block of code for errors.

The `except` block lets you handle the error.

The `else` block lets you execute code when there is no error.

The `finally` block lets you execute code, regardless of the result of the try- and except blocks.

The `try` block will generate an exception, because `x` is not defined:

Input:

```
try:
    print(x)
except:
    print("An exception occurred")
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:\asif-office-work\python_code\class_2.py =====
An exception occurred
|
```

Print one message if the try block raises a `NameError` and another for other errors:

Input:

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

```
===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Variable x is not defined
|
```

In this example, the `try` block does not generate any error:

Input:

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Hello
Nothing went wrong
|
```

Input:

```
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")|
```

Output:

```
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 14:12:15) [MSC v.1929 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

===== RESTART: D:\asif-office-work\python_code\class_2.py =====
Something went wrong
The 'try except' is finished
|
```

Raise an exception

As a Python developer you can choose to throw an exception if a condition occurs.

To throw (or raise) an exception, use the **raise** keyword.

```
x = -1

if x < 0:
    raise Exception("Sorry, no numbers below zero")
```

```
Traceback (most recent call last):
  File "demo_ref_keyword_raise.py", line 4, in <module>
    raise Exception("Sorry, no numbers below zero")
Exception: Sorry, no numbers below zero
```

The **raise** keyword is used to raise an exception.

You can define what kind of error to raise, and the text to print to the user.

```
x = "hello"
```

```
if not type(x) is int:  
    raise TypeError("Only integers are allowed")
```

```
Traceback (most recent call last):  
  File "demo_ref_keyword_raise2.py", line 4, in <module>  
    raise TypeError("Only integers are allowed")  
TypeError: Only integers are allowed
```

Project

THE END